

```
01 # ===== FILE: bio_logic_debugger/__init__.py =====
02 """让郳城燃烧 — Bio-Logic Debugger: 育种逻辑验证系统"""
03
04 __version__ = "0.2.10"
05 __author__ = "势魏延"
06
07 # ===== FILE: bio_logic_debugger/paths.py =====
08 """用户数据目录。社区缓存、用户知识、权重都放这里，不写进安装包。"""
09 from __future__ import annotations
10
11 import os
12 from pathlib import Path
13
14
15 def user_data_dir() -> Path:
16     raw = os.environ.get("BIO_LOGIC_HOME", "").strip()
17     if raw:
18         path = Path(raw).expanduser()
19     else:
20         path = Path.home() / ".bio-logic-debugger"
21     path.mkdir(parents=True, exist_ok=True)
22     return path
23
24
25 def community_dir() -> Path:
26     path = user_data_dir() / "community"
27     path.mkdir(parents=True, exist_ok=True)
28     return path
29
30 # ===== FILE: bio_logic_debugger/core/domain.py =====
31 """
32 Bio-Logic Debugger — 核心领域模型
33
34 定义生物学逻辑约束系统的骨架数据结构。
35 所有模块以此为基础进行构建，保证可扩展性与类型安全。
36 """
37 from __future__ import annotations
38
39 import hashlib
40 import json
41 from dataclasses import dataclass, field
42 from datetime import datetime
43 from enum import Enum, auto
44 from typing import Any
45
46
47 # =====
48 # 基础枚举
49 # =====
50
```

```

01 class CorrelationType(Enum):
02     """两性状之间的关联类型"""
03     POSITIVE = auto()          # 正相关: A↑ → B↑
04     NEGATIVE = auto()         # 负相关: A↑ → B↓
05     CURVILINEAR = auto()      # 曲线相关: 存在最优区间
06     THRESHOLD = auto()        # 阈值依赖: 超过某值后关系翻转
07     EPISTATIC = auto()        # 上位效应: 依赖第三个性状的存在
08     TRADE_OFF = auto()        # 权衡: 不可兼得, 必须取舍
09
10
11 class ConstraintSeverity(Enum):
12     """约束违反的严重等级"""
13     FATAL = auto()            # 生理上不可能, 绝对死胡同
14     SEVERE = auto()           # 极难突破, 历史上极少成功
15     WARNING = auto()          # 有冲突但可能通过特殊手段缓解
16     INFO = auto()             # 需要注意, 信息性提示
17
18
19 class EvidenceLevel(Enum):
20     """证据等级: 知识的确信度"""
21     CONFIRMED = auto()         # 多篇独立文献+多年田间验证
22     STRONG = auto()            # 有可靠文献支持
23     SUGGESTED = auto()         # 单篇文献或初步观察
24     ANECDOTAL = auto()         # 经验总结, 未严格验证
25     HYPOTHETICAL = auto()      # 理论推导, 待验证
26
27
28 class ConstraintScope(Enum):
29     """约束的适用范围"""
30     UNIVERSAL = auto()         # 适用于所有作物
31     GENUS = auto()             # 适用于某个属 (如 Oryza)
32     SPECIES = auto()           # 适用于某个种
33     SUBSPECIES = auto()        # 适用于某个亚种/生态型
34     CUSTOM = auto()            # 用户自定义
35
36
37 # -----
38 # 性状定义
39 # -----
40
41 @dataclass
42 class Trait:
43     """
44     一个可测量的生物学性状。
45
46     这是系统中最基本的原子单位。一切约束、关联、反模式
47     最终都落在性状上。
48
49     Attributes:
50         id: 唯一标识符, 如 'rice.yield.per_plant'

```

```

01     name: 人类可读名称, 如 '单株产量'
02     description: 详细描述, 包括测量方法
03     category: 分类, 如 '产量', '品质', '抗病', '株型', '生理'
04     unit: 测量单位
05     typical_range: 该性状在目标物种中的典型取值范围 [min, max]
06     tags: 标签, 用于快速分类检索
07     """
08     id: str
09     name: str
10     description: str
11     category: str
12     unit: str = ""
13     typical_range: tuple[float | None, float | None] = (None, None)
14     tags: list[str] = field(default_factory=list)
15     species: str = "通用"
16     confidence: float = 1.0
17     # SES 1-9 级: 1 为优/抗, 9 为劣/感, 越大越差
18     higher_is_better: bool = True
19     aliases: list[str] = field(default_factory=list)
20
21
22 # -----
23 # 性状关联
24 # -----
25
26 @dataclass
27 class CorrelationEvidence:
28     """支撑一条关联规则的证据记录"""
29     level: EvidenceLevel
30     source: str          # 文献引用或来源说明
31     description: str      # 证据具体内容
32     url: str = ""        # 链接
33     year: int = 0        # 发表年份
34
35
36 @dataclass
37 class TraitCorrelation:
38     """
39     两个性状之间的关联关系。
40
41     这是约束系统的核心: 如果育种目标同时要求一对负相关性状
42     都达到极值, 引擎就会在这里触发警告。
43
44     Attributes:
45         trait_a, trait_b: 两个性状的 ID
46         corr_type: 关联类型
47         strength: 关联强度 [-1, 1], 负值表示负相关
48         confidence: 置信度 [0, 1]
49         mechanism: 生理机制解释——育种家关心的“为什么”
50         conditions: 该关联成立的前提条件 (如“C3植物”)

```

```

01     evidence: 支撑证据列表
02     antagonistic_threshold: 拮抗阈值, 当双方需求超过此值时触发
03     """
04     trait_a: str
05     trait_b: str
06     corr_type: CorrelationType
07     strength: float          # -1.0 ~ 1.0
08     confidence: float = 1.0  # 0.0 ~ 1.0
09     mechanism: str = ""      # 生理机制解释
10     conditions: list[str] = field(default_factory=list)
11     evidence: list[CorrelationEvidence] = field(default_factory=list)
12     antagonistic_threshold: float = 0.3 # 超过此强度视为"拮抗"
13
14     def is_antagonistic(self) -> bool:
15         """是否为拮抗关系（负相关且强度超过阈值）"""
16         return self.corr_type in (
17             CorrelationType.NEGATIVE,
18             CorrelationType.TRADE_OFF,
19         ) and abs(self.strength) >= self.antagonistic_threshold
20
21
22 # -----
23 # 约束规则
24 # -----
25
26 @dataclass
27 class BiologicalConstraint:
28     """
29     一条生物学约束规则。
30
31     比 TraitCorrelation 更灵活: 可以表达"如果A超过X且B低于Y则违反"
32     这样的复合条件, 而不只是两两关联。
33
34     Attributes:
35         id: 唯一标识
36         name: 规则名称
37         description: 规则描述
38         severity: 违反后果的严重等级
39         scope: 适用范围
40         species: 适用的物种
41         condition_expr: 条件表达式（结构化的判定逻辑）
42         consequence: 违反的生理后果描述
43         confidence: 置信度
44         evidence: 证据列表
45         tags: 标签
46     """
47     id: str
48     name: str
49     description: str
50     severity: ConstraintSeverity

```

```

01     scope: ConstraintScope = ConstraintScope.UNIVERSAL
02     species: str = "通用"
03     condition_expr: str = ""          # 条件表达式, 详见 engine 文档
04     consequence: str = ""
05     confidence: float = 1.0
06     evidence: list[CorrelationEvidence] = field(default_factory=list)
07     tags: list[str] = field(default_factory=list)
08
09
10 # -----
11 # 反模式
12 # -----
13
14 @dataclass
15 class FailedApproach:
16     """历史上尝试过但失败的育种策略"""
17     description: str
18     reason_failed: str
19     year_range: tuple[int, int] = (0, 0)
20     reference: str = ""
21
22
23 @dataclass
24 class AntiPattern:
25     """
26     一个“反模式”——被历史反复验证的育种死胡同。
27
28     它与 BiologicalConstraint 的区别在于：约束是普适的生理法则，
29     而反模式是经验的、历史的、叙事性的。它告诉育种家
30     “这条路以前有人走过，全都失败了，原因如下”。
31
32     Attributes:
33         id: 唯一标识
34         name: 反模式名称, 如“高产低质的陷阱”
35         description: 详细描述
36         trigger_traits: 触发此反模式的性状组合
37         severity: 严重等级
38         historical_examples: 历史案例
39         failed_approaches: 已尝试的失败策略
40         alternative_directions: 推荐的替代方向
41         mechanism: 背后的生理机制
42         confidence: 置信度
43         evidence: 支撑证据
44         tags: 标签
45     """
46     id: str
47     name: str
48     description: str
49     trigger_traits: list[str]          # 相关的 trait id 列表
50     # trait_id -> better/worse/any; 空则只看性状是否出现

```

```

01     trigger_intents: dict[str, str] = field(default_factory=dict)
02     severity: ConstraintSeverity = ConstraintSeverity.WARNING
03     historical_examples: list[str] = field(default_factory=list)
04     failed_approaches: list[FailedApproach] = field(default_factory=list)
05     alternative_directions: list[str] = field(default_factory=list)
06     mechanism: str = ""
07     confidence: float = 1.0
08     evidence: list[CorrelationEvidence] = field(default_factory=list)
09     tags: list[str] = field(default_factory=list)
10     species: str = "通用"
11
12     @property
13     def signature(self) -> str:
14         """生成反模式的特征签名, 用于快速匹配"""
15         sorted_traits = sorted(self.trigger_traits)
16         raw = f"{self.species}:{','.join(sorted_traits)}"
17         return hashlib.sha256(raw.encode()).hexdigest()[:16]
18
19
20 # -----
21 # 育种目标
22 # -----
23
24 @dataclass
25 class TraitTarget:
26     """用户对某个性状的育种目标"""
27     trait_id: str
28     desired_value: float | None = None    # 目标数值
29     direction: str = ">="                # >=, <=, ==, range
30     range_min: float | None = None
31     range_max: float | None = None
32     priority: int = 5                    # 1-10, 10最高
33     note: str = ""
34
35
36 @dataclass
37 class EnvironmentVariable:
38     """
39     环境变量: 不是育种性状, 但是约束表达式会引用。
40     例如干旱程度、温度、施氮量。
41     """
42     id: str
43     name: str
44     description: str
45     value_type: str = "number" # number / enum
46     unit: str = ""
47     enum_values: list[str] = field(default_factory=list)
48
49
50 # 内置约束会引用这些 id; UI / CLI 需要能填, 校验也要把它们算进合法变量。

```

```

01 ENVIRONMENT_VARIABLES: list[EnvironmentVariable] = [
02     EnvironmentVariable(
03         id="drought_severity",
04         name="干旱程度",
05         description="田间干旱胁迫等级",
06         value_type="enum",
07         enum_values=["mild", "moderate", "severe"],
08     ),
09     EnvironmentVariable(
10         id="temperature",
11         name="环境温度",
12         description="关键生育期环境温度",
13         value_type="number",
14         unit="° C",
15     ),
16     EnvironmentVariable(
17         id="nitrogen_rate",
18         name="施氮量",
19         description="氮肥施用量",
20         value_type="number",
21         unit="kg/ha",
22     ),
23 ]
24
25 ENVIRONMENT_VAR_IDS: frozenset[str] = frozenset(v.id for v in ENVIRONMENT_VARIABLES)
26
27
28 @dataclass
29 class BreedingGoal:
30     """
31     用户定义的育种目标。
32
33     这是系统的输入：用户说"我想要A、B、C三个性状分别达到什么水平",
34     系统以此为基础做验证。
35     """
36     name: str
37     species: str = "通用"
38     targets: list[TraitTarget] = field(default_factory=list)
39     context: str = "" # 额外的背景描述
40     environment: dict[str, Any] = field(default_factory=dict)
41     created_at: datetime = field(default_factory=datetime.now)
42
43     def add_target(self, target: TraitTarget) -> None:
44         self.targets.append(target)
45
46     def trait_ids(self) -> list[str]:
47         return [t.trait_id for t in self.targets]
48
49
50 #

```

```

01 # 验证结果
02 #
03
04 @dataclass
05 class Violation:
06     """一次约束违反的记录"""
07     constraint_id: str
08     severity: ConstraintSeverity
09     title: str
10     description: str          # 问题描述
11     mechanism: str            # 生理机制解释
12     narrative: str            # 可读的"顾问式"叙事
13     involved_traits: list[str]
14     suggestion: str = ""      # 建议
15     source: str = "rule"      # rule / anti_pattern / llm
16
17     def __post_init__(self):
18         # 类型安全: 确保 severity 不会意外被覆盖为 string
19         if isinstance(self.severity, str):
20             self.severity = ConstraintSeverity[self.severity.upper()]
21
22
23 @dataclass
24 class ValidationReport:
25     """
26     一份完整的育种目标验证报告。
27
28     这是系统的输出——不仅告诉用户通不通过，
29     还给出完整的叙事性解释。
30     """
31     goal: BreedingGoal
32     passed: bool = True
33     violations: list[Violation] = field(default_factory=list)
34     matched_anti_patterns: list[AntiPattern] = field(default_factory=list)
35     suggestions: list[str] = field(default_factory=list)
36     llm_comment: str = ""
37     generated_at: datetime = field(default_factory=datetime.now)
38
39     def add_violation(self, v: Violation) -> None:
40         self.violations.append(v)
41         if v.severity == ConstraintSeverity.FATAL:
42             self.passed = False
43
44     def verdict(self) -> str:
45         """给非专业用户的结论。只有 FATAL 才否定推进。"""
46         if any(v.severity == ConstraintSeverity.FATAL for v in self.violations):
47             return "不建议按原目标推进"
48         if (
49             any(v.severity == ConstraintSeverity.SEVERE for v in self.violations)
50             or self.matched_anti_patterns

```

```

01         ):
02             return "谨慎推进"
03         return "可以推进"
04
05     def summary(self) -> dict:
06         """返回报告的摘要字典，便于序列化"""
07         return {
08             "goal": self.goal.name,
09             "passed": self.passed,
10             "total_violations": len(self.violations),
11             "fatal": sum(1 for v in self.violations if v.severity == ConstraintSeverity.FATAL),
12             "severe": sum(1 for v in self.violations if v.severity == ConstraintSeverity.SEVERE),
13             "warnings": sum(1 for v in self.violations if v.severity == ConstraintSeverity.WARNING),
14             "infos": sum(1 for v in self.violations if v.severity == ConstraintSeverity.INFO),
15             "anti_patterns_matched": len(self.matched_anti_patterns),
16             "suggestions_count": len(self.suggestions),
17         }
18
19     def narrative(self) -> str:
20         """生成完整的顾问式叙事报告"""
21         lines = [f"# 育种目标验证报告: {self.goal.name}"]
22         lines.append(f"物种: {self.goal.species}")
23         lines.append(f"结论: {self.verdict()}")
24         lines.append("")
25
26         if self.violations:
27             lines.append("## 发现的问题")
28             for v in sorted(self.violations, key=lambda x: x.severity.value):
29                 tag = {
30                     ConstraintSeverity.FATAL: "? 致命",
31                     ConstraintSeverity.SEVERE: "? 严重",
32                     ConstraintSeverity.WARNING: "? 警告",
33                     ConstraintSeverity.INFO: "? 提示",
34                 }.get(v.severity, "□ 未知")
35                 lines.append(f"\n### [{tag}] {v.title}")
36                 lines.append(f"{v.narrative}")
37
38         if self.suggestions:
39             lines.append("\n## 建议方向")
40             for i, s in enumerate(self.suggestions, 1):
41                 lines.append(f"{i}. {s}")
42
43         if self.llm_comment:
44             lines.append(f"\n## LLM 分析意见\n{self.llm_comment}")
45
46         lines.append("")
47         lines.append("——")
48         lines.append(f"报告生成时间: {self.generated_at.isoformat()}")
49         return "\n".join(lines)
50

```

```

01     def to_json(self, indent: int = 2) -> str:
02         """序列化为 JSON"""
03         def _serialize(obj: Any) -> Any:
04             if isinstance(obj, Enum):
05                 return obj.name
06             if isinstance(obj, datetime):
07                 return obj.isoformat()
08             if isinstance(obj, TraitTarget):
09                 return {
10                     "trait_id": obj.trait_id,
11                     "desired_value": obj.desired_value,
12                     "direction": obj.direction,
13                     "range_min": obj.range_min,
14                     "range_max": obj.range_max,
15                     "priority": obj.priority,
16                     "note": obj.note,
17                 }
18             if isinstance(obj, Violation):
19                 return {
20                     "constraint_id": obj.constraint_id,
21                     "severity": obj.severity.name,
22                     "title": obj.title,
23                     "description": obj.description,
24                     "mechanism": obj.mechanism,
25                     "narrative": obj.narrative,
26                     "involved_traits": obj.involved_traits,
27                     "suggestion": obj.suggestion,
28                     "source": obj.source,
29                 }
30             if isinstance(obj, AntiPattern):
31                 return {
32                     "id": obj.id,
33                     "name": obj.name,
34                     "description": obj.description,
35                     "severity": obj.severity.name,
36                     "trigger_traits": obj.trigger_traits,
37                     "alternative_directions": obj.alternative_directions,
38                 }
39             if hasattr(obj, "__dataclass_fields__"):
40                 return {k: _serialize(v) for k, v in obj.__dict__.items()}
41         return str(obj)
42
43     return json.dumps(_serialize(self), ensure_ascii=False, indent=indent)
44
45 # ===== FILE: bio_logic_debugger/core/expr.py =====
46 """
47 约束条件表达式。
48
49 语法（与知识库里现有 condition_expr 对齐）：
50     $trait_id >= 120

```

```

01  $a > 25 AND $b > 80
02  $heading_days < 65
03  NOT ($x <= 3)
04  $flag = 'severe'
05
06  不执行 Python、不用 eval。缺绑定的 $变量时视为无法判定，约束不触发。
07  """
08  from __future__ import annotations
09
10  import math
11  from dataclasses import dataclass
12  from typing import Any, Union
13
14
15  class ExprError(ValueError):
16      pass
17
18
19  @dataclass(frozen=True)
20  class Binding:
21      """
22      变量绑定。
23
24      - 点值: lo == hi == value (或 kind=enum 的字符串)
25      - 区间: >= v → [v, +∞); <= v → (-∞, v]; range → [min, max]
26      direction 用来决定「推向危险区用 may, 反向用 must」。
27      """
28      value: Any = None
29      lo: float | None = None
30      hi: float | None = None
31      direction: str = ""
32      kind: str = "point" # point / interval / enum
33
34      def numeric_interval(self) -> tuple[float, float] | None:
35          if self.kind == "enum":
36              return None
37          if self.kind == "point" and self.value is not None:
38              try:
39                  v = float(self.value)
40              except (TypeError, ValueError):
41                  return None
42              return v, v
43          lo = self.lo if self.lo is not None else -math.inf
44          hi = self.hi if self.hi is not None else math.inf
45          return lo, hi
46
47
48  @dataclass(frozen=True)
49  class Literal:
50      value: float | str

```

```
01
02
03 @dataclass(frozen=True)
04 class Var:
05     name: str
06
07
08 @dataclass(frozen=True)
09 class Compare:
10     left: Var
11     op: str
12     right: Literal
13
14
15 @dataclass(frozen=True)
16 class Not:
17     inner: "Node"
18
19
20 @dataclass(frozen=True)
21 class BoolOp:
22     op: str # AND / OR
23     items: tuple["Node", ...]
24
25
26 Node = Union[Literal, Var, Compare, Not, BoolOp]
27
28
29 def referenced_ids(node: Node) -> list[str]:
30     if isinstance(node, Var):
31         return [node.name]
32     if isinstance(node, Compare):
33         return [node.left.name]
34     if isinstance(node, Not):
35         return referenced_ids(node.inner)
36     if isinstance(node, BoolOp):
37         names: list[str] = []
38         for item in node.items:
39             for n in referenced_ids(item):
40                 if n not in names:
41                     names.append(n)
42     return names
43
44
45
46 def parse(expr: str) -> Node:
47     tokens = _tokenize(expr)
48     if not tokens:
49         raise ExprError("空表达式")
50     parser = _Parser(tokens)
```

```

01     node = parser.parse_or()
02     if parser.i < len(tokens):
03         raise ExprError(f"表达式末尾有多余记号: {tokens[parser.i]}")
04     return node
05
06
07 def binding_from_target(desired_value: Any, direction: str,
08                         range_min: float | None = None,
09                         range_max: float | None = None) -> Binding | None:
10     """把育种目标方向收成区间绑定。无数值则无法判定。"""
11     if direction == "range":
12         if range_min is None and range_max is None:
13             return None
14         return Binding(
15             value=range_min if range_min is not None else range_max,
16             lo=range_min,
17             hi=range_max,
18             direction="range",
19             kind="interval",
20         )
21     if desired_value is None:
22         return None
23     if isinstance(desired_value, str):
24         return Binding(value=desired_value, direction=direction, kind="enum")
25     try:
26         v = float(desired_value)
27     except (TypeError, ValueError):
28         return Binding(value=desired_value, direction=direction, kind="enum")
29     if direction in (">=", ">"):
30         return Binding(value=v, lo=v, hi=None, direction=direction, kind="interval")
31     if direction in ("<=", "<"):
32         return Binding(value=v, lo=None, hi=v, direction=direction, kind="interval")
33     return Binding(value=v, lo=v, hi=v, direction=direction or "=", kind="point")
34
35
36 def evaluate(node: Node, env: dict[str, Any]) -> bool:
37     """env 缺变量时抛 KeyError, 由调用方当成「无法判定」。"""
38     if isinstance(node, Compare):
39         if node.left.name not in env:
40             raise KeyError(node.left.name)
41         return _eval_compare(env[node.left.name], node.op, node.right.value)
42     if isinstance(node, Not):
43         return not evaluate(node.inner, env)
44     if isinstance(node, BoolOp):
45         if node.op == "AND":
46             return all(evaluate(item, env) for item in node.items)
47         return any(evaluate(item, env) for item in node.items)
48     raise ExprError(f"无法求值的节点: {type(node).__name__}")
49
50

```

```

01 def try_evaluate(expr: str, env: dict[str, Any]) -> tuple[bool, str]:
02     """
03     返回 (是否成立, 原因)。
04     原因非空表示跳过: 空表达式、语法错误、缺变量。
05     """
06     text = (expr or "").strip()
07     if not text:
08         return False, "empty"
09     try:
10         node = parse(text)
11     except ExprError as e:
12         return False, f"syntax: {e}"
13     try:
14         return evaluate(node, env), ""
15     except KeyError as e:
16         return False, f"unbound: {e.args[0]}"
17
18
19 def _eval_compare(raw: Any, op: str, right: Any) -> bool:
20     if isinstance(raw, Binding):
21         return _eval_binding(raw, op, right)
22     return _compare(raw, op, right)
23
24
25 def _direction_agrees(direction: str, op: str) -> bool:
26     pushing_high = direction in (">=", ">")
27     pushing_low = direction in ("<=", "<")
28     danger_high = op in (">", ">=")
29     danger_low = op in ("<", "<=")
30     return (pushing_high and danger_high) or (pushing_low and danger_low)
31
32
33 def _interval_compare(lo: float, hi: float, op: str, right: Any) -> tuple[bool, bool]:
34     """返回 (must_true, may_true)。"""
35     if op in ("=", "==", "!="):
36         try:
37             rv = float(right)
38         except (TypeError, ValueError):
39             return False, False
40         inside = lo <= rv <= hi
41         if op == "!=":
42             must = rv < lo or rv > hi
43             may = not (lo == hi == rv)
44             return must, may
45         must = lo == hi == rv
46         return must, inside
47     try:
48         rv = float(right)
49     except (TypeError, ValueError):
50         return False, False

```

```

01     if op == ">":
02         return lo > rv, hi > rv
03     if op == ">=":
04         return lo >= rv, hi >= rv
05     if op == "<":
06         return hi < rv, lo < rv
07     if op == "<=":
08         return hi <= rv, lo <= rv
09     raise ExprError(f"未知比较符: {op}")
10
11
12 def _eval_binding(binding: Binding, op: str, right: Any) -> bool:
13     if binding.kind == "enum" or (
14         binding.kind != "interval" and isinstance(binding.value, str)
15     ):
16         return _compare(binding.value, op, right)
17     interval = binding.numeric_interval()
18     if interval is None:
19         return _compare(binding.value, op, right)
20     lo, hi = interval
21     must, may = _interval_compare(lo, hi, op, right)
22     if op in ("=", "=="):
23         return must
24     if op == "!=":
25         return may
26     if binding.direction == "range":
27         return may
28     if _direction_agrees(binding.direction, op):
29         return may
30     return must
31
32
33 def _compare(left: Any, op: str, right: Any) -> bool:
34     if op in ("=", "=="):
35         return left == right
36     if op == "!=":
37         return left != right
38     try:
39         lv = float(left)
40         rv = float(right)
41     except (TypeError, ValueError):
42         return False
43     if op == ">":
44         return lv > rv
45     if op == ">=":
46         return lv >= rv
47     if op == "<":
48         return lv < rv
49     if op == "<=":
50         return lv <= rv

```

```

01     raise ExprError(f"未知比较符: {op}")
02
03
04 def _tokenize(expr: str) -> list[tuple[str, str]]:
05     s = expr.strip()
06     i = 0
07     out: list[tuple[str, str]] = []
08     n = len(s)
09     while i < n:
10         ch = s[i]
11         if ch.isspace():
12             i += 1
13             continue
14         if s.startswith(">=", i) or s.startswith("<=", i) or s.startswith("!= ", i) or s.startswith("== ", i):
15             out.append(("op", s[i : i + 2]))
16             i += 2
17             continue
18         if ch in "><=":
19             out.append(("op", ch))
20             i += 1
21             continue
22         if ch in "()":
23             out.append(("paren", ch))
24             i += 1
25             continue
26         if ch in "\\":
27             j = i + 1
28             while j < n and s[j] != ch:
29                 j += 1
30             if j >= n:
31                 raise ExprError("字符串未闭合")
32             out.append(("string", s[i + 1 : j]))
33             i = j + 1
34             continue
35         if ch == "$":
36             j = i + 1
37             if j >= n or not (s[j].isalpha() or s[j] == "_"):
38                 raise ExprError("$ 后需要标识符")
39             j += 1
40             while j < n and (s[j].isalnum() or s[j] == "_"):
41                 j += 1
42             out.append(("var", s[i + 1 : j]))
43             i = j
44             continue
45         if (
46             ch.isdigit()
47             or (ch == "." and i + 1 < n and s[i + 1].isdigit())
48             or (ch == "-" and i + 1 < n and (s[i + 1].isdigit() or s[i + 1] == "."))
49         ):
50             j = i + 1 if ch == "-" else i

```

```

01         if j < n and s[j] == ".":
02             j += 1
03         while j < n and s[j].isdigit():
04             j += 1
05         if j < n and s[j] == ".":
06             j += 1
07             while j < n and s[j].isdigit():
08                 j += 1
09         out.append(("number", s[i:j]))
10         i = j
11         continue
12     if ch.isalpha() or ch == "_":
13         j = i + 1
14         while j < n and (s[j].isalnum() or s[j] == "_"):
15             j += 1
16         word = s[i:j].upper()
17         if word in ("AND", "OR", "NOT"):
18             out.append(("kw", word))
19         else:
20             raise ExprError(f"未知关键字: {s[i:j]}")
21         i = j
22         continue
23     raise ExprError(f"非法字符: {ch}")
24 return out
25
26
27 class _Parser:
28     def __init__(self, tokens: list[tuple[str, str]]):
29         self.tokens = tokens
30         self.i = 0
31
32     def peek(self) -> tuple[str, str] | None:
33         if self.i >= len(self.tokens):
34             return None
35         return self.tokens[self.i]
36
37     def eat(self, kind: str | None = None, value: str | None = None) -> tuple[str, str]:
38         tok = self.peek()
39         if tok is None:
40             raise ExprError("表达式不完整")
41         if kind and tok[0] != kind:
42             raise ExprError(f"期望 {kind}, 得到 {tok}")
43         if value and tok[1] != value:
44             raise ExprError(f"期望 {value}, 得到 {tok[1]}")
45         self.i += 1
46         return tok
47
48     def parse_or(self) -> Node:
49         items = [self.parse_and()]
50         while self.peek() == ("kw", "OR"):

```

```

01         self.eat("kw", "OR")
02         items.append(self.parse_and())
03     if len(items) == 1:
04         return items[0]
05     return BoolOp("OR", tuple(items))
06
07     def parse_and(self) -> Node:
08         items = [self.parse_not()]
09         while self.peak() == ("kw", "AND"):
10             self.eat("kw", "AND")
11             items.append(self.parse_not())
12         if len(items) == 1:
13             return items[0]
14         return BoolOp("AND", tuple(items))
15
16     def parse_not(self) -> Node:
17         if self.peak() == ("kw", "NOT"):
18             self.eat("kw", "NOT")
19             return Not(self.parse_not())
20         return self.parse_atom()
21
22     def parse_atom(self) -> Node:
23         tok = self.peak()
24         if tok is None:
25             raise ExprError("表达式不完整")
26         if tok == ("paren", "("):
27             self.eat("paren", "(")
28             node = self.parse_or()
29             self.eat("paren", ")")
30             return node
31         if tok[0] != "var":
32             raise ExprError("比较左侧必须是 $变量")
33         name = self.eat("var")[1]
34         op = self.eat("op")[1]
35         rhs = self.peak()
36         if rhs is None:
37             raise ExprError("比较缺少右值")
38         if rhs[0] == "number":
39             self.eat("number")
40             value: float | str = float(rhs[1])
41         elif rhs[0] == "string":
42             self.eat("string")
43             value = rhs[1]
44         else:
45             raise ExprError("比较右值必须是数字或字符串")
46         return Compare(Var(name), op, Literal(value))
47
48 # ===== FILE: bio_logic_debugger/core/anti_pattern.py =====
49 """
50 反模式匹配引擎

```

```

01
02 反模式（AntiPattern）不等同于约束规则。约束规则是普适的生理法则，
03 而反模式是经验的、历史的、叙事性的——它告诉育种家“这条路以前有人走过，
04 全都失败了，原因如下”。
05
06 匹配策略：
07     1. 精确匹配：育种目标中的性状组合直接触发某个反模式
08     2. 部分匹配：性状组合与反模式部分重叠，给出提示
09     3. 语义匹配：通过 LLM 判断育种方向是否与已知反模式在语义上相似
10 """
11 from __future__ import annotations
12
13 from dataclasses import dataclass, field
14
15 from .domain import AntiPattern, BreedingGoal
16
17
18 @dataclass
19 class AntiPatternMatch:
20     """
21     一次反模式匹配的结果。
22
23     Attributes:
24         anti_pattern: 匹配到的反模式
25         match_type: 匹配类型（精确/部分/语义）
26         matched_traits: 实际匹配到的性状
27         missing_traits: 反模式中有但育种目标未提及的性状
28         score: 匹配度 [0, 1]
29     """
30     anti_pattern: AntiPattern
31     match_type: str # exact / partial / semantic
32     matched_traits: list[str] = field(default_factory=list)
33     missing_traits: list[str] = field(default_factory=list)
34     score: float = 0.0
35
36
37 class AntiPatternMatcher:
38     """
39     反模式匹配器。
40
41     使用多策略匹配：先做精确匹配（高性能），
42     再做部分匹配（召回），
43     语义匹配交给外部 LLM。
44     """
45
46     def __init__(self, anti_patterns: list[AntiPattern] | None = None):
47         self._patterns: dict[str, AntiPattern] = {} # id -> pattern
48         self._trait_index: dict[str, list[str]] = {} # trait_id -> [pattern_id, ...]
49         self._confidence_threshold: float = 0.3
50

```

```

01         if anti_patterns:
02             for p in anti_patterns:
03                 self.register(p)
04
05     def register(self, pattern: AntiPattern) -> None:
06         """注册一个反模式到索引"""
07         self._patterns[pattern.id] = pattern
08         for tid in pattern.trigger_traits:
09             if tid not in self._trait_index:
10                 self._trait_index[tid] = []
11                 self._trait_index[tid].append(pattern.id)
12
13     def register_many(self, patterns: list[AntiPattern]) -> None:
14         for p in patterns:
15             self.register(p)
16
17     def unregister(self, pattern_id: str) -> None:
18         """移除一个反模式"""
19         pattern = self._patterns.pop(pattern_id, None)
20         if pattern:
21             for tid in pattern.trigger_traits:
22                 if tid in self._trait_index:
23                     self._trait_index[tid] = [
24                         pid for pid in self._trait_index[tid] if pid != pattern_id
25                     ]
26
27     def match(self, goal: BreedingGoal) -> list[AntiPatternMatch]:
28         """
29         对育种目标执行反模式匹配。
30
31         返回按匹配度排序的匹配结果列表。
32         """
33         goal_traits = set(goal.trait_ids())
34         results: list[AntiPatternMatch] = []
35
36         # 策略1: 直接遍历所有反模式 (小型知识库适用)
37         for pattern in self._patterns.values():
38             pattern_traits = set(pattern.trigger_traits)
39             matched = goal_traits & pattern_traits
40             missing = pattern_traits - goal_traits
41
42             if not matched or not pattern_traits:
43                 continue
44
45             # 覆盖率: 反模式触发性状被目标命中的比例。
46             # 不用 Jaccard: 额外目标性状不应把已命中的死胡同打下去。
47             coverage = len(matched) / len(pattern_traits)
48             score = coverage * pattern.confidence
49
50             if score < self._confidence_threshold:

```

```

01         continue
02
03         if matched == pattern_traits:
04             match_type = "exact"
05         elif coverage >= 0.5:
06             match_type = "partial"
07         else:
08             match_type = "weak"
09
10         results.append(AntiPatternMatch(
11             anti_pattern=pattern,
12             match_type=match_type,
13             matched_traits=sorted(matched),
14             missing_traits=sorted(missing),
15             score=round(score, 3),
16         ))
17
18     # 按匹配度从高到低排序
19     results.sort(key=lambda r: (-r.score, r.match_type))
20     return results
21
22 def quick_check(self, goal: BreedingGoal) -> list[AntiPatternMatch]:
23     """
24     快速检查：仅返回高置信度的精确/高度部分匹配。
25     用于初始筛选阶段，避免信息过载。
26     """
27     all_matches = self.match(goal)
28     return [
29         m for m in all_matches
30         if m.match_type == "exact"
31         or (m.match_type == "partial" and m.score >= 0.5)
32     ]
33
34 @property
35 def count(self) -> int:
36     return len(self._patterns)
37
38 def all_patterns(self) -> list[AntiPattern]:
39     return list(self._patterns.values())
40
41 def trait_coverage(self) -> dict[str, int]:
42     """返回覆盖每个性状的反模式数量"""
43     return {t: len(ids) for t, ids in self._trait_index.items()}
44
45 # ===== FILE: bio_logic_debugger/core/engine.py =====
46 """
47 Bio-Logic Debugger — 约束验证引擎
48
49 引擎是整个系统的核心编排器。它接收一个育种目标，
50 依次经过多层验证管线，最终生成一份完整的验证报告。

```

```

01
02 验证管线（按执行顺序）：
03     1. 基础检查层：性状是否存在、数值是否在合理范围内
04     2. 关联检查层：逐对检查目标性状之间的关联，发现拮抗关系
05     3. 约束规则层：检查是否符合已知的生物学约束
06     4. 反模式匹配层：检查是否触发了历史反模式
07     5. （可选）LLM 推理层：调用大模型做更深层的生物合理性分析
08 """
09 from __future__ import annotations
10
11 import logging
12 from dataclasses import dataclass, field
13 from typing import Any, Callable, Optional
14
15 from .anti_pattern import AntiPatternMatcher
16 from .domain import (
17     AntiPattern,
18     BiologicalConstraint,
19     BreedingGoal,
20     ConstraintSeverity,
21     CorrelationType,
22     Trait,
23     TraitCorrelation,
24     TraitTarget,
25     ValidationReport,
26     Violation,
27 )
28 from .expr import Binding, ExprError, binding_from_target, parse, referenced_ids, try_evaluate
29
30 logger = logging.getLogger(__name__)
31 _UNSET = object()
32
33
34 # -----
35 # 验证管线接口
36 # -----
37
38 @dataclass
39 class ValidationContext:
40     """
41     验证上下文，在管线各层之间传递数据。
42
43     每一层可以向 context 写入中间结果，
44     供后续层或最终报告生成使用。
45     """
46     goal: BreedingGoal
47     trait_map: dict[str, Trait] = field(default_factory=dict)
48     violations: list[Violation] = field(default_factory=list)
49     matched_anti_patterns: list[AntiPattern] = field(default_factory=list)
50     suggestions: list[str] = field(default_factory=list)

```

```

01     llm_comment: str = ""
02
03     @property
04     def is_fatal(self) -> bool:
05         return any(
06             v.severity == ConstraintSeverity.FATAL
07             for v in self.violations
08         )
09
10
11 ValidationLayer = Callable[[ValidationContext], ValidationContext]
12
13
14 # -----
15 # 引擎主体
16 # -----
17
18 class BioLogicEngine:
19     """
20     生物逻辑验证引擎的主类。
21
22     用法：
23         engine = BioLogicEngine()
24         engine.register_trait(rice_trait)
25         engine.register_correlation(rice_corr)
26         report = engine.validate(goal)
27         print(report.narrative())
28     """
29
30     def __init__(self):
31         self._traits: dict[str, Trait] = {}
32         self._alias_to_id: dict[str, str] = {}
33         self._correlations: list[TraitCorrelation] = []
34         self._constraints: list[BiologicalConstraint] = []
35         self._anti_patterns: AntiPatternMatcher = AntiPatternMatcher()
36         self._layers: list[ValidationLayer] = []
37         self._llm_callback: Optional[Callable] = None
38
39         # 注册默认验证层
40         self._register_default_layers()
41
42     def _register_default_layers(self) -> None:
43         self._layers = [
44             self._layer_basic_check,
45             self._layer_correlation_check,
46             self._layer_constraint_check,
47             self._layer_anti_pattern_match,
48         ]
49
50     # ----- 注册 API -----

```

```

01
02     def register_trait(self, trait: Trait) -> None:
03         self._traits[trait.id] = trait
04         self._alias_to_id[trait.id] = trait.id
05         for alias in trait.aliases or []:
06             if alias:
07                 self._alias_to_id[alias] = trait.id
08
09     def canonical_id(self, trait_id: str) -> str:
10         return self._alias_to_id.get(trait_id, trait_id)
11
12     def register_traits(self, traits: list[Trait]) -> None:
13         for t in traits:
14             self.register_trait(t)
15
16     def register_correlation(self, corr: TraitCorrelation) -> None:
17         corr.trait_a = self.canonical_id(corr.trait_a)
18         corr.trait_b = self.canonical_id(corr.trait_b)
19         self._correlations.append(corr)
20
21     def register_correlations(self, corrs: list[TraitCorrelation]) -> None:
22         for corr in corrs:
23             self.register_correlation(corr)
24
25     def register_constraint(self, constraint: BiologicalConstraint) -> None:
26         constraint.condition_expr = self._rewrite_expr_ids(constraint.condition_expr or "")
27         self._constraints.append(constraint)
28
29     def register_constraints(self, constraints: list[BiologicalConstraint]) -> None:
30         for constraint in constraints:
31             self.register_constraint(constraint)
32
33     def _rewrite_expr_ids(self, expr: str) -> str:
34         if not expr:
35             return expr
36         for alias in sorted(self._alias_to_id, key=len, reverse=True):
37             canon = self._alias_to_id[alias]
38             if alias == canon:
39                 continue
40             expr = expr.replace("$" + alias, "$" + canon)
41         return expr
42
43     def register_anti_pattern(self, pattern: AntiPattern) -> None:
44         pattern.trigger_traits = [self.canonical_id(t) for t in pattern.trigger_traits]
45         if pattern.trigger_intents:
46             pattern.trigger_intents = {
47                 self.canonical_id(k): v for k, v in pattern.trigger_intents.items()
48             }
49         self._anti_patterns.register(pattern)
50

```

```

01     def register_anti_patterns(self, patterns: list[AntiPattern]) -> None:
02         for pattern in patterns:
03             self.register_anti_pattern(pattern)
04
05     def register_layer(self, layer: ValidationLayer, index: int | None = None) -> None:
06         """注册自定义验证层"""
07         if index is None:
08             self._layers.append(layer)
09         else:
10             self._layers.insert(index, layer)
11
12     def set_llm_callback(self, callback: Callable | None) -> None:
13         """设置 LLM 推理回调; 传 None 关闭。界面验证应走 validate(llm_layer=...)。"""
14         self._llm_callback = callback
15
16     def set_confidence(self, entity_type: str, entity_id: str, value: float) -> bool:
17         """按类型和 id 改置信度。关联 id 为排序后的 a_b。"""
18         value = float(value)
19         kind = entity_type.rstrip("s")
20         if kind == "trait":
21             trait = self.get_trait(entity_id)
22             if trait is None:
23                 return False
24             trait.confidence = value
25             return True
26         if kind == "correlation":
27             parts = entity_id.split("__")
28             if len(parts) != 2:
29                 return False
30             a = self.canonical_id(parts[0])
31             b = self.canonical_id(parts[1])
32             for corr in self._correlations:
33                 if {corr.trait_a, corr.trait_b} == {a, b}:
34                     corr.confidence = value
35                     return True
36             return False
37         if kind == "constraint":
38             for constraint in self._constraints:
39                 if constraint.id == entity_id:
40                     constraint.confidence = value
41                     return True
42             return False
43         return False
44
45     def get_trait(self, trait_id: str) -> Optional[Trait]:
46         return self._traits.get(self.canonical_id(trait_id))
47
48     def trait_name(self, trait_id: str) -> str:
49         return self._trait_name(trait_id)
50

```

```

01     def iter_traits(self):
02         return self._traits.values()
03
04     def iter_correlations(self):
05         return list(self._correlations)
06
07     def iter_anti_patterns(self):
08         return self._anti_patterns.all_patterns()
09
10     def iter_constraints(self):
11         return list(self._constraints)
12
13     def trait_ids(self) -> list[str]:
14         return sorted(self._traits.keys())
15
16     # ----- 验证管线 -----
17
18     def validate(self, goal: BreedingGoal, llm_layer: Any = _UNSET) -> ValidationReport:
19         """
20         执行完整的验证管线。
21
22         流程：
23         1. 构建验证上下文
24         2. 依次执行各验证层
25         3. （可选）调用 LLM 推理
26         4. 生成最终报告
27         """
28         from dataclasses import replace
29         work = replace(
30             goal,
31             targets=[replace(t, trait_id=self.canonical_id(t.trait_id)) for t in goal.targets],
32         )
33         ctx = ValidationContext(
34             goal=work,
35             trait_map={k: v for k, v in self._traits.items()},
36         )
37
38         # 各层都跑完：FATAL 不应吞掉约束和反模式。
39         for layer in self._layers:
40             ctx = layer(ctx)
41
42         callback = self._llm_callback if llm_layer is _UNSET else llm_layer
43         if callback:
44             try:
45                 ctx = callback(ctx)
46             except Exception as e:
47                 logger.warning(f"LLM 推理失败: {e}")
48                 ctx.llm_comment = f"[LLM 推理异常: {e}]"
49
50         return ValidationReport(

```

```

01         goal=goal,
02         passed=not any(
03             v.severity == ConstraintSeverity.FATAL for v in ctx.violations
04         ),
05         violations=ctx.violations,
06         matched_anti_patterns=ctx.matched_anti_patterns,
07         suggestions=ctx.suggestions,
08         llm_comment=ctx.llm_comment,
09     )
10
11 # ----- 默认验证层实现 -----
12
13 @staticmethod
14 def _layer_basic_check(ctx: ValidationContext) -> ValidationContext:
15     """
16     基础检查层:
17     - 目标性状是否在知识库中
18     - 目标数值是否在合理范围内
19     """
20     for target in ctx.goal.targets:
21         trait = ctx.trait_map.get(target.trait_id)
22         if trait is None:
23             ctx.suggestions.append(
24                 f"性状 '{target.trait_id}' 不在当前知识库中，"
25                 "将跳过与此性状相关的所有检查。建议先补充该性状的定义。"
26             )
27             continue
28
29     # 检查数值范围
30     lo, hi = trait.typical_range
31     if target.desired_value is not None and lo is not None and hi is not None:
32         if target.desired_value < lo or target.desired_value > hi:
33             # 低置信度的 trait，范围警告降级
34             range_severity = (
35                 ConstraintSeverity.INFO
36                 if trait.confidence < 0.5
37                 else ConstraintSeverity.WARNING
38             )
39             ctx.violations.append(Violation(
40                 constraint_id=f"range_check. {trait.id}",
41                 severity=range_severity,
42                 title=f"目标值超出 {trait.name} 的典型范围",
43                 description=(
44                     f"'{trait.name}' 的典型范围是 [{lo}, {hi}] {trait.unit}，"
45                     f"而育种目标设置为 {target.desired_value}"
46                 ),
47                 mechanism="数值超出物种的生理极限范围",
48                 narrative=(
49                     f"关于「{trait.name}」：您设定的目标 ({target.desired_value}) "
50                     f"超过典型范围 ({lo:.2f} ~ {hi:.2f})。\\n"

```

```

01         f"这并不意味着绝对不可能，但如果该数值超出了物种已知的生理极限，"
02         f"可能需要引入远缘种质资源才能实现。"
03     ),
04     involved_traits=[trait.id],
05     suggestion=(
06         f"请确认：是否确实需要 {target.desired_value}? "
07         f"如果这是打破现有纪录的目标，建议分阶段实现。"
08     ),
09     source="rule",
10 ))
11
12 return ctx
13
14 def _layer_correlation_check(self, ctx: ValidationContext) -> ValidationContext:
15     """
16     关联检查层：
17     逐对检查用户目标中的性状之间是否存在拮抗关系。
18     """
19     goal_traits = set(ctx.goal.trait_ids())
20
21     for corr in self._correlations:
22         if corr.trait_a not in goal_traits or corr.trait_b not in goal_traits:
23             continue
24
25         target_a = self._find_target(ctx.goal, corr.trait_a)
26         target_b = self._find_target(ctx.goal, corr.trait_b)
27
28         tname_a = self._trait_name(corr.trait_a)
29         tname_b = self._trait_name(corr.trait_b)
30
31         if corr.is_antagonistic():
32             trait_a = ctx.trait_map.get(corr.trait_a)
33             trait_b = ctx.trait_map.get(corr.trait_b)
34             user_wants_both_high = (
35                 self._wants_high(target_a, trait_a)
36                 and self._wants_high(target_b, trait_b)
37             )
38
39             if not user_wants_both_high and corr.correlation_type == CorrelationType.TRADE_OFF:
40                 continue
41
42             if user_wants_both_high:
43                 effective = abs(corr.strength) * corr.confidence
44                 if effective < 0.2:
45                     continue
46                 # 负相关是权衡，不是生理不可能。最高只给 WARNING。
47                 narrative_parts = [
48                     f"在「{tname_a}」和「{tname_b}」之间存在一个已知的{self._corr_type_label(corr)}关系",
49                     f" (r = {corr.strength:.2f}, 置信度 {corr.confidence:.2f}, 有效强度 {effective:.2f})。",
50                 ]

```

```

01         if corr.mechanism:
02             narrative_parts.append(f"\n\n背后的生理机制: {corr.mechanism}")
03
04         narrative_parts.append(
05             "\n\n同时追求两者走高时, 一方的提升可能被另一方拖累。"
06             "这是统计关联, 不是「生理上不可能」。"
07         )
08
09         ctx.violations.append(Violation(
10             constraint_id=f"corr. {corr.trait_a}. {corr.trait_b}",
11             severity=ConstraintSeverity.WARNING,
12             title=f"拮抗关系: {tname_a} □ {tname_b}",
13             description=(
14                 f"'{tname_a}' 与 '{tname_b}' 之间存在 {self._corr_type_label(corr)}, "
15                 f"强度 {corr.strength:.2f}"
16             ),
17             mechanism=corr.mechanism or "未知机制",
18             narrative="".join(narrative_parts),
19             involved_traits=[corr.trait_a, corr.trait_b],
20             suggestion=(
21                 "建议: 如果可能, 尝试降低其中之一目标值。"
22                 "或者寻找是否存在打破该连锁的特殊种质资源。"
23             ),
24             source="rule",
25         ))
26
27     elif corr.corr_type == CorrelationType.POSITIVE:
28         trait_a = ctx.trait_map.get(corr.trait_a)
29         trait_b = ctx.trait_map.get(corr.trait_b)
30         better_a = self._wants_high(target_a, trait_a)
31         better_b = self._wants_high(target_b, trait_b)
32         chasing_a = self._is_chasing(target_a)
33         chasing_b = self._is_chasing(target_b)
34         if not chasing_a or not chasing_b:
35             continue
36         effective = abs(corr.strength) * corr.confidence
37         if effective < 0.2:
38             continue
39         same_way = better_a == better_b
40         if same_way:
41             title = f"正相关联: {tname_a} □ {tname_b}"
42             narrative = (
43                 f"「{tname_a}」和「{tname_b}」正相关"
44                 f"(r = {corr.strength:.2f}, 有效强度 {effective:.2f})。"
45                 f"两边同向追极端时, 一方会带动另一方, 风险会一起放大。"
46             )
47             suggestion = "同向加压前先看哪一侧先碰到生理上限。"
48         else:
49             title = f"正相关顺风: {tname_a} □ {tname_b}"
50             narrative = (

```

```

01         f"「{tname_a}」和「{tname_b}」正相关"
02         f" (r = {corr.strength:.2f}, 有效强度 {effective:.2f})。"
03         f"一边追优、一边追劣时，这条关联是顺风，不是拮抗。"
04     )
05     suggestion = "可把这条当作协同，不必按权衡去拆目标。"
06     ctx.violations.append(Violation(
07         constraint_id=f"corr_pos. {corr.trait_a}. {corr.trait_b}",
08         severity=ConstraintSeverity.INFO,
09         title=title,
10         description=(
11             f"'{tname_a}' 与 '{tname_b}' 正相关，强度 {corr.strength:.2f}"
12         ),
13         mechanism=corr.mechanism or "未知机制",
14         narrative=narrative,
15         involved_traits=[corr.trait_a, corr.trait_b],
16         suggestion=suggestion,
17         source="rule",
18     ))
19
20     elif corr.corr_type == CorrelationType.CURVILINEAR:
21         # 曲线关系：存在最优区间，过高或过低都不好
22         ctx.violations.append(Violation(
23             constraint_id=f"corr_curve. {corr.trait_a}. {corr.trait_b}",
24             severity=ConstraintSeverity.INFO,
25             title=f"曲线关系提示：{tname_a} 与 {tname_b}",
26             description="两者之间存在曲线相关，存在最优配比区间",
27             mechanism=corr.mechanism or "未知机制",
28             narrative=(
29                 f"「{tname_a}」和「{tname_b}」之间不是简单的线性关系，"
30                 f"而是曲线相关。这意味着存在一个最优配比区间，"
31                 f"过高或过低都会导致综合表现下降。\\n\\n"
32                 f"建议不要同时追求两者最大化，而是要找到最佳平衡点。"
33             ),
34             involved_traits=[corr.trait_a, corr.trait_b],
35             suggestion="使用响应面法（RSM）寻找最优配比区间。",
36             source="rule",
37         ))
38
39     return ctx
40
41 def _layer_constraint_check(self, ctx: ValidationContext) -> ValidationContext:
42     """
43     约束规则检查层：
44     用 condition_expr 对照目标数值求值。缺变量或语法错误则跳过，不误报。
45     """
46     env = self._goal_env(ctx.goal)
47     for constraint in self._constraints:
48         fired, skip_reason = try_evaluate(constraint.condition_expr, env)
49         if skip_reason or not fired:
50             if skip_reason and skip_reason.startswith("syntax:"):

```

```

01
02     def __init__(self, reasoner: LLMReasoner):
03         self.reasoner = reasoner
04
05     def match(
06         self,
07         goal: BreedingGoal,
08         candidates: list,
09     ) -> list[tuple[str, float, str]]:
10         """
11         对候选反模式进行语义匹配。
12
13         返回: [(反模式ID, 匹配度, 解释), ...]
14         """
15         if not candidates:
16             return []
17
18         patterns_text = "\n\n".join(
19             f"ID: {p.id}\n名称: {p.name}\n描述: {p.description[:200]}"
20             for p in candidates
21         )
22
23         targets_str = "\n".join(
24             f" - {t.trait_id}: {'>=' if t.direction == '>=' else '<=' if t.direction == '<=' else '==' } {t.de
25             for t in goal.targets if t.desired_value
26         )
27
28         prompt = f"""育种目标:
29 名称: {goal.name}
30 目标性状:
31 {targets_str}
32
33 以下是候选的历史反模式（已知的育种死胡同）:
34
35 {patterns_text}
36
37 请判断: 这个育种目标在语义上与哪些反模式相似?
38 对于每个匹配的反模式, 给出:
39 1. ID
40 2. 匹配度 (0.0 - 1.0)
41 3. 匹配的理由
42
43 以JSON格式输出: [{"id": "...", "score": 0.0, "reason": "..."}]
44 只输出 JSON, 不要其他内容。"""
45
46         messages = [
47             {"role": "system", "content": "你是一位育种专家助手。请严格按 JSON 格式输出分析结果。"},
48             {"role": "user", "content": prompt},
49         ]
50

```

```

01         try:
02             import httpx
03             headers = {
04                 "Authorization": f"Bearer {self.reasoner.config.api_key}",
05                 "Content-Type": "application/json",
06             }
07             payload = {
08                 "model": self.reasoner.config.model,
09                 "messages": messages,
10                 "temperature": 0.3,
11                 "max_tokens": 1024,
12             }
13             resp = httpx.post(
14                 f"{self.reasoner.config.base_url.rstrip('/')}/chat/completions",
15                 json=payload,
16                 headers=headers,
17                 timeout=60,
18             )
19             resp.raise_for_status()
20             data = resp.json()
21             content = data["choices"][0]["message"]["content"]
22
23             # 提取 JSON
24             content = content.strip()
25             if content.startswith("```"):
26                 content = content.split("\n", 1)[1].rsplit("\n", 1)[0]
27             if content.endswith("```"):
28                 content = content[:-3]
29
30             results = json.loads(content)
31             return [
32                 (r["id"], float(r["score"]), r["reason"])
33                 for r in results
34                 if r["score"] >= 0.4
35             ]
36         except Exception as e:
37             logger.warning(f"语义匹配失败: {e}")
38         return []
39
40 # ===== FILE: bio_logic_debugger/ui/runtime.py =====
41 """引擎缓存、侧边栏、启动同步。"""
42 from __future__ import annotations
43
44 import streamlit as st
45
46 from bio_logic_debugger import __version__
47 from bio_logic_debugger.core.engine import BioLogicEngine
48 from bio_logic_debugger.knowledge import knowledge_store
49 from bio_logic_debugger.knowledge.knowledge_store import load_and_merge
50 from bio_logic_debugger.knowledge.weight_store import apply_weights_to_engine

```

```

01
02
03 @st.cache_resource
04 def get_engine() -> BioLogicEngine:
05     engine = BioLogicEngine()
06     try:
07         traits, correlations, constraints, anti_patterns = load_and_merge()
08         engine.register_traits(traits)
09         engine.register_correlations(correlations)
10         engine.register_constraints(constraints)
11         engine.register_anti_patterns(anti_patterns)
12     except Exception as e:
13         try:
14             from bio_logic_debugger.knowledge.knowledge_store import load_built_in_objects
15             traits, correlations, constraints, anti_patterns = load_built_in_objects()
16             engine.register_traits(traits)
17             engine.register_correlations(correlations)
18             engine.register_constraints(constraints)
19             engine.register_anti_patterns(anti_patterns)
20             st.warning(f"知识库合并失败，使用内置兜底: {e}")
21         except Exception as e2:
22             st.error(f"内置知识也无法加载: {e2}")
23     try:
24         apply_weights_to_engine(engine)
25     except Exception:
26         pass
27     return engine
28
29
30 def trait_label(engine: BioLogicEngine, trait_id: str) -> str:
31     t = engine.get_trait(trait_id)
32     if not t:
33         return trait_id
34     label = t.name
35     if t.typical_range[0] is not None:
36         label += f" ({t.typical_range[0]}~{t.typical_range[1]} {t.unit})"
37     return label
38
39
40 def boot() -> None:
41     # 启动不再自动拉取社区库，避免远程 JSON 静默覆盖判决。
42     if "_version_notified" not in st.session_state:
43         st.session_state._version_notified = True
44         st.toast(f"当前版本 v{__version__}", icon="📄")
45
46
47 def render_sidebar(engine: BioLogicEngine) -> str:
48     st.sidebar.markdown(
49         "<h1 style='font-size: 1.5rem;'>? 让郾城燃烧</h1>",
50         unsafe_allow_html=True,

```

```

01     )
02     st.sidebar.markdown("**Bio-Logic Debugger**")
03     st.sidebar.caption(f"v{_version_}")
04     st.sidebar.markdown("——")
05
06     page = st.sidebar.radio(
07         "导航",
08         ["育种目标验证", "性状浏览器", "反模式库", "约束规则", "? 文献与知识库"],
09         label_visibility="collapsed",
10     )
11
12     st.sidebar.markdown("——")
13     st.sidebar.caption("用生物学逻辑在播种前筛掉注定失败的育种方向。")
14     st.sidebar.caption("社区知识不会启动自动覆盖，需手动点「检查更新」。")
15
16     st.sidebar.markdown("——")
17     st.sidebar.caption("? 知识库状态")
18     with st.sidebar:
19         trait_count = len(list(engine.iter_traits()))
20         corr_count = len(engine.iter_correlations())
21         ap_count = len(engine.iter_anti_patterns())
22         last_sync = knowledge_store.get_last_sync_time()
23         has_community = knowledge_store.has_community_data()
24
25         st.caption("☐ 已同步" if has_community else "☐ 内置模式（未同步）")
26         st.caption(f"同步于 {last_sync}" if last_sync else "从未同步")
27         st.caption(f"性状 {trait_count} / 关联 {corr_count} / 反模式 {ap_count}")
28         try:
29             from bio_logic_debugger.knowledge.paper_search import load_seen_papers
30             st.caption(f"已检索 {len(load_seen_papers())} 篇论文")
31         except Exception:
32             pass
33
34         if st.button("? 检查更新", use_container_width=True, key="sidebar_sync"):
35             with st.spinner("检查社区知识库..."):
36                 ok = knowledge_store.sync_from_community()
37                 if ok:
38                     st.cache_resource.clear()
39                     st.toast("☐ 社区知识库已更新，引擎已重新加载", icon="?")
40                     st.rerun()
41                 else:
42                     st.toast("☐☐ 同步失败，请检查网络", icon="☐☐")
43     return page
44
45 # ===== FILE: bio_logic_debugger/ui/pages/validate.py =====
46 from __future__ import annotations
47
48 import os
49
50 import streamlit as st

```

```

01
02 from bio_logic_debugger.core.domain import (
03     ENVIRONMENT_VARIABLES,
04     BreedingGoal,
05     ConstraintSeverity,
06     TraitTarget,
07 )
08 from bio_logic_debugger.core.engine import BioLogicEngine
09 from bio_logic_debugger.ui.runtime import trait_label
10
11
12 def _load_demo_goal(engine: BioLogicEngine) -> None:
13     """软著截图用: BIO_LOGIC_DEMO=1 时预填高秆+高抗示例并跑一次验证。"""
14     if os.environ.get("BIO_LOGIC_DEMO") != "1":
15         return
16     if "goal_targets" not in st.session_state:
17         st.session_state.goal_targets = [
18             {"trait_id": "rice_plant_height", "direction": ">=", "value": 130.0, "priority": 8},
19             {"trait_id": "rice_lodging_resistance", "direction": "<=", "value": 2.0, "priority": 8},
20         ]
21     if "last_report" not in st.session_state:
22         goal = BreedingGoal(name="高秆高抗示例", species="水稻")
23         for tgt in st.session_state.goal_targets:
24             goal.add_target(TraitTarget(
25                 trait_id=tgt["trait_id"],
26                 desired_value=tgt["value"],
27                 direction=tgt["direction"],
28                 priority=tgt["priority"],
29             ))
30         st.session_state.last_report = engine.validate(goal)
31
32
33 def render(engine: BioLogicEngine) -> None:
34     _load_demo_goal(engine)
35     st.title("育种目标验证")
36     st.markdown("设定育种目标，系统会检查拮抗关系、生理约束和已知反模式。")
37
38     col_left, col_right = st.columns([3, 2])
39
40     with col_left:
41         st.subheader("目标设定")
42         goal_name = st.text_input("育种目标名称", "未命名品种")
43         available_traits = engine.trait_ids()
44         categories = set(t.category for t in engine.iter_traits())
45
46         if "goal_targets" not in st.session_state:
47             st.session_state.goal_targets = []
48
49         with st.container(border=True):
50             cat_filter = st.selectbox(

```

```

01         "分类筛选", ["全部"] + sorted(categories), key="cat_filter"
02     )
03     filtered = []
04     for tid in available_traits:
05         trait = engine.get_trait(tid)
06         if trait is None:
07             continue
08         if cat_filter == "全部" or trait.category == cat_filter:
09             filtered.append(tid)
10     trait_id = st.selectbox(
11         "性状", filtered,
12         format_func=lambda tid: trait_label(engine, tid),
13     )
14     dir_col, val_col, pri_col = st.columns([1, 2, 1])
15     with dir_col:
16         direction = st.selectbox("方向", [ ">=", "<=", "==" ], key="direction")
17     with val_col:
18         desired_value = st.number_input("目标值", value=0.0, step=0.1, format="%.2f")
19     with pri_col:
20         priority = st.number_input("优先级", min_value=1, max_value=10, value=5)
21
22     if st.button("□ 添加目标", use_container_width=True):
23         st.session_state.goal_targets.append({
24             "trait_id": trait_id,
25             "direction": direction,
26             "value": desired_value,
27             "priority": priority,
28         })
29         st.rerun()
30
31     if st.session_state.goal_targets:
32         st.markdown("**已设定的目标: **")
33         for i, tgt in enumerate(st.session_state.goal_targets):
34             tname = engine.trait_name(tgt["trait_id"])
35             trait = engine.get_trait(tgt["trait_id"])
36             range_str = ""
37             if trait and trait.typical_range[0] is not None:
38                 range_str = f"    typical: {trait.typical_range[0]}~{trait.typical_range[1]} {trait.unit}"
39             cols = st.columns([0.1, 0.7, 0.1])
40             with cols[0]:
41                 st.markdown(f"**{i+1}.**")
42             with cols[1]:
43                 st.markdown(
44                     f"**{tname}**  {tgt['direction']} {tgt['value']}"
45                     f"    <优先级 {tgt['priority']}>"
46                     f"<span style='color:#888;font-size:0.8em;'> {range_str}</span>",
47                     unsafe_allow_html=True,
48                 )
49             with cols[2]:
50                 if st.button("□", key=f"del_{i}"):

```

```

01         st.session_state.goal_targets.pop(i)
02         st.rerun()
03
04     if st.button("? 清空所有", type="secondary"):
05         st.session_state.goal_targets = []
06         st.rerun()
07
08     st.divider()
09     env_values: dict = {}
10     with st.expander("? 环境条件（可选）"):
11         st.caption("约束里的干旱程度 / 温度 / 施氮量在这里填，不填则相关约束跳过。")
12         for var in ENVIRONMENT_VARIABLES:
13             if var.value_type == "enum":
14                 options = ["（不设）"] + list(var.enum_values)
15                 chosen = st.selectbox(var.name, options, key=f"env_{var.id}")
16                 if chosen != "（不设）":
17                     env_values[var.id] = chosen
18             else:
19                 raw = st.text_input(
20                     f"{var.name}（{var.unit}）" if var.unit else var.name,
21                     value="",
22                     key=f"env_{var.id}",
23                 )
24                 raw = raw.strip()
25                 if raw:
26                     try:
27                         env_values[var.id] = float(raw)
28                     except ValueError:
29                         st.warning(f"{var.name} 需要数字")
30
31     with st.expander("□□ LLM 深度分析（可选）"):
32         llm_enabled = st.checkbox("启用 LLM 分析", value=False)
33         api_key = st.text_input("API Key", type="password", placeholder="sk-... 或设置 BIO_LLM_API_KEY")
34         base_url = st.text_input("Base URL", placeholder="https://api.deepseek.com/v1")
35         model = st.text_input("模型名", placeholder="deepseek-chat")
36
37     if st.button("? 运行验证", type="primary", use_container_width=True):
38         goal = BreedingGoal(
39             name=goal_name,
40             species="水稻",
41             environment=env_values,
42         )
43         for tgt in st.session_state.goal_targets:
44             goal.add_target(TraitTarget(
45                 trait_id=tgt["trait_id"],
46                 desired_value=tgt["value"],
47                 direction=tgt["direction"],
48                 priority=tgt["priority"],
49             ))
50         llm_layer = None

```

```

01         if llm_enabled:
02             from bio_logic_debugger.llm.reasoner import LLMConfig, LLMReasoner
03             reasoner = LLMReasoner(config=LLMConfig.from_ui(
04                 api_key=api_key,
05                 base_url=base_url,
06                 model=model,
07             ))
08             llm_layer = reasoner.as_validation_layer(engine)
09         with st.spinner("验证中..."):
10             st.session_state.last_report = engine.validate(
11                 goal, llm_layer=llm_layer,
12             )
13         st.rerun()
14
15     with col_right:
16         st.subheader("验证结果")
17         if "last_report" not in st.session_state:
18             st.info("左侧设定目标后点击「运行验证」")
19         else:
20             report = st.session_state.last_report
21             verdict = report.verdict()
22             if verdict == "可以推进":
23                 st.success(f"✅ {verdict}")
24             elif verdict == "谨慎推进":
25                 st.warning(f"⚠️ {verdict}")
26             else:
27                 st.error(f"❌ {verdict}")
28             summary = report.summary()
29             mc1, mc2, mc3, mc4 = st.columns(4)
30             mc1.metric("致命", summary["fatal"], border=True)
31             mc2.metric("严重", summary["severe"], border=True)
32             mc3.metric("警告", summary["warnings"], border=True)
33             mc4.metric("提示", summary["infos"], border=True)
34             if summary["anti_patterns_matched"]:
35                 st.markdown(f"❌❌ 匹配到 {summary['anti_patterns_matched']} 个反模式")
36
37     if "last_report" in st.session_state:
38         report = st.session_state.last_report
39         if report.violations:
40             st.subheader("? 详细问题列表")
41             severity_order = {
42                 ConstraintSeverity.FATAL: 0,
43                 ConstraintSeverity.SEVERE: 1,
44                 ConstraintSeverity.WARNING: 2,
45                 ConstraintSeverity.INFO: 3,
46             }
47             tag_map = {
48                 ConstraintSeverity.FATAL: ("?", "致命", "#ff4b4b"),
49                 ConstraintSeverity.SEVERE: ("?", "严重", "#ff922b"),
50                 ConstraintSeverity.WARNING: ("?", "警告", "#fcc419"),

```

```

01         ConstraintSeverity.INFO: ("?", "提示", "#339af0"),
02     }
03     for v in sorted(report.violations, key=lambda x: severity_order.get(x.severity, 99)):
04         icon, tag, _color = tag_map.get(v.severity, ("□", "未知", "#888"))
05         expanded = v.severity in (ConstraintSeverity.FATAL, ConstraintSeverity.SEVERE)
06         with st.expander(f"{icon} [{tag}] {v.title}", expanded=expanded):
07             st.markdown(v.description)
08             if v.involved_traits:
09                 names = " ".join(f"`{engine.trait_name(tid)}`" for tid in v.involved_traits)
10                 st.markdown("**涉及性状:** " + names)
11             if v.mechanism:
12                 st.markdown(f"**机制:** {v.mechanism}")
13             st.markdown("——")
14             st.markdown(v.narrative)
15             if v.suggestion:
16                 st.info(f"? {v.suggestion}")
17     if report.suggestions:
18         st.subheader("? 建议方向")
19         for i, s in enumerate(report.suggestions, 1):
20             st.markdown(f"{i}. {s}")
21     if report.llm_comment:
22         st.subheader("? LLM 分析意见")
23         st.markdown(report.llm_comment)
24
25 # ===== FILE: bio_logic_debugger/ui/pages/browser.py =====
26 from __future__ import annotations
27
28 import streamlit as st
29
30 from bio_logic_debugger.core.domain import CorrelationType
31 from bio_logic_debugger.core.engine import BioLogicEngine
32
33
34 def render(engine: BioLogicEngine) -> None:
35     st.title("? 性状浏览器")
36     st.markdown("浏览所有可用性状及其关联关系。")
37
38     categories = sorted(set(t.category for t in engine.iter_traits()))
39     selected_cat = st.selectbox("按分类筛选", ["全部"] + categories)
40     traits_to_show = [
41         t for t in engine.iter_traits()
42         if selected_cat == "全部" or t.category == selected_cat
43     ]
44     by_cat: dict[str, list] = {}
45     for t in traits_to_show:
46         by_cat.setdefault(t.category, []).append(t)
47
48     correlations = engine.iter_correlations()
49     for cat, ts in sorted(by_cat.items()):
50         st.subheader(f"? {cat}")

```

```

01     cols = st.columns(3)
02     for i, t in enumerate(sorted(ts, key=lambda x: x.id)):
03         with cols[i % 3]:
04             with st.container(border=True):
05                 st.markdown(f"**{t.name}**")
06                 st.caption(f"`{t.id}`")
07                 if t.typical_range[0] is not None:
08                     st.markdown(
09                         f"范围: **{t.typical_range[0]} ~ {t.typical_range[1]}** {t.unit}"
10                     )
11                 else:
12                     st.markdown(f"单位: {t.unit}")
13                 st.caption(t.description)
14                 related = [c for c in correlations if t.id in (c.trait_a, c.trait_b)]
15                 if related:
16                     with st.expander(f"关联 ({len(related)})"):
17                         for c in related:
18                             other = c.trait_b if c.trait_a == t.id else c.trait_a
19                             other_name = engine.trait_name(other)
20                             tag = (
21                                 "? 拮抗" if c.is_antagonistic()
22                                 else "? 权衡" if c.corr_type == CorrelationType.TRADE_OFF
23                                 else "? 正相关" if c.corr_type == CorrelationType.POSITIVE
24                                 else "? 相关"
25                             )
26                             st.markdown(f"{tag} **{other_name}** (r={c.strength})")
27                             mech = c.mechanism[:80] + "..." if len(c.mechanism) > 80 else c.mechanism
28                             st.caption(mech)
29
30 # ===== FILE: bio_logic_debugger/ui/pages/anti_patterns.py =====
31 from __future__ import annotations
32
33 import streamlit as st
34
35 from bio_logic_debugger.core.domain import ConstraintSeverity
36 from bio_logic_debugger.core.engine import BioLogicEngine
37
38
39 def render(engine: BioLogicEngine) -> None:
40     st.title("? 反模式库")
41     st.markdown("历史上反复验证的育种死胡同。")
42     severity_filter = st.selectbox("按等级筛选", ["全部", "严重", "警告", "提示"])
43
44     tag_map = {
45         ConstraintSeverity.FATAL: ("?", "致命"),
46         ConstraintSeverity.SEVERE: ("?", "严重"),
47         ConstraintSeverity.WARNING: ("?", "警告"),
48         ConstraintSeverity.INFO: ("?", "提示"),
49     }
50     for ap in engine.iter_anti_patterns():

```

```

01         if severity_filter == "严重" and ap.severity not in (
02             ConstraintSeverity.SEVERE, ConstraintSeverity.FATAL
03         ):
04             continue
05         if severity_filter == "警告" and ap.severity != ConstraintSeverity.WARNING:
06             continue
07         icon, tag = tag_map.get(ap.severity, ("□", ""))
08         with st.expander(f"{icon} **[{tag}] {ap.name}**"):
09             st.markdown(ap.description)
10             names = " ".join(f"`{engine.trait_name(t)}`" for t in ap.trigger_traits)
11             st.markdown("**触发性状: ** " + names)
12             if ap.mechanism:
13                 st.markdown(f"**生理机制: ** {ap.mechanism}")
14             if ap.historical_examples:
15                 st.markdown("**历史案例: **")
16                 for ex in ap.historical_examples:
17                     st.markdown(f"- {ex}")
18             if ap.failed_approaches:
19                 st.markdown("**失败的尝试: **")
20                 for fa in ap.failed_approaches:
21                     st.markdown(f"- {fa.description} —— _{fa.reason_failed}_")
22             if ap.alternative_directions:
23                 st.success("**推荐的替代方向: **")
24                 for i, alt in enumerate(ap.alternative_directions, 1):
25                     st.markdown(f"[i]. {alt}")
26             st.caption(f"置信度: {ap.confidence:.0%}")
27
28 # ===== FILE: bio_logic_debugger/ui/pages/constraints.py =====
29 from __future__ import annotations
30
31 import streamlit as st
32
33 from bio_logic_debugger.core.domain import ConstraintSeverity
34 from bio_logic_debugger.core.engine import BioLogicEngine
35
36
37 def render(engine: BioLogicEngine) -> None:
38     st.title("? 生物学约束规则")
39     st.markdown(
40         "FATAL: 生理极限。SEVERE: 极难突破。WARNING: 已知冲突, 可能缓解。"
41     )
42     tag_map = {
43         ConstraintSeverity.FATAL: ("?", "致命"),
44         ConstraintSeverity.SEVERE: ("?", "严重"),
45         ConstraintSeverity.WARNING: ("?", "警告"),
46         ConstraintSeverity.INFO: ("?", "提示"),
47     }
48     for constraint in engine.iter_constraints():
49         icon, tag = tag_map.get(constraint.severity, ("□", ""))
50         with st.container(border=True):

```

```

01         cols = st.columns([0.05, 1])
02         with cols[0]:
03             st.markdown(f"**{icon}**")
04         with cols[1]:
05             st.markdown(f"**[{tag}] {constraint.name}**")
06             st.markdown(constraint.description)
07             st.markdown(f"适用范围: `{constraint.scope.name}` / {constraint.species}")
08             if constraint.consequence:
09                 st.markdown(f"后果: _{constraint.consequence}_")
10             if constraint.condition_expr:
11                 st.code(constraint.condition_expr, language="text")
12             st.caption(f"置信度: {constraint.confidence:.0%}")
13
14 # ===== FILE: bio_logic_debugger/ui/pages/literature.py =====
15 from __future__ import annotations
16
17 import streamlit as st
18
19 from bio_logic_debugger.core.engine import BioLogicEngine
20 from bio_logic_debugger.knowledge import knowledge_store
21 from bio_logic_debugger.knowledge.weight_store import load_weights as load_user_weights
22 from bio_logic_debugger.knowledge.weight_store import merge_slider_overrides
23 from bio_logic_debugger.knowledge.weight_store import save_weights as save_user_weights
24
25
26 def render(engine: BioLogicEngine) -> None:
27     st.title("? 文献与知识库")
28     st.markdown("上传论文、分析图表、管理知识库。")
29
30     tab1, tab2, tab3 = st.tabs(["? 导入文献分析", "? 知识库管理", "? 贡献指南"])
31
32     # ----- Tab 1: 导入文献分析 -----
33
34     with tab1:
35         st.subheader("导入文献分析")
36         st.markdown("从论文中提取性状、关联和约束，审核后导入知识库。")
37
38         # 来源选择
39         source_method = st.radio(
40             "论文来源",
41             ["上传 PDF", "DOI / 标题搜索"],
42             horizontal=True,
43         )
44
45         pdf_bytes = None
46         paper_meta = {}
47
48         if source_method == "上传 PDF":
49             uploaded = st.file_uploader(
50                 "上传 PDF 文件", type=["pdf"],

```

```

01         help="支持标准的学术论文 PDF",
02     )
03     if uploaded:
04         pdf_bytes = uploaded.read()
05         st.success(f"已上传: {uploaded.name} ({len(pdf_bytes) // 1024} KB)")
06         # 提取文本
07         from bio_logic_debugger.knowledge.pdf_parser import extract_text
08         with st.spinner("提取文本中..."):
09             raw_text = extract_text(pdf_bytes)
10             st.info(f"提取到 {len(raw_text)} 字符")
11             st.session_state.paper_raw_text = raw_text
12
13     else:
14         col1, col2 = st.columns([3, 1])
15         with col1:
16             search_input = st.text_input(
17                 "DOI 或论文标题",
18                 placeholder="10.1007/s00122-021-03867-w 或输入标题...",
19             )
20         with col2:
21             search_btn = st.button("? 检索", use_container_width=True)
22
23     if search_btn and search_input:
24         from bio_logic_debugger.knowledge.doi_fetcher import (
25             fetch_by_doi, search_by_title,
26         )
27         with st.spinner("检索中..."):
28             # 检测是否为 DOI
29             if search_input.startswith("10."):
30                 paper_meta = fetch_by_doi(search_input) or {}
31             else:
32                 paper_meta = search_by_title(search_input) or {}
33
34             if paper_meta:
35                 st.success(f"找到: {paper_meta.get('title', '未知')}")
36                 st.session_state.paper_meta = paper_meta
37                 st.session_state.paper_raw_text = paper_meta.get("abstract", "")
38             else:
39                 st.warning("未找到相关论文")
40
41     # 显示已检索的元数据
42     if "paper_meta" in st.session_state and st.session_state.paper_meta:
43         pm = st.session_state.paper_meta
44         with st.container(border=True):
45             st.markdown(f"**{pm.get('title')}**")
46             cols = st.columns(4)
47             cols[0].caption(f"作者: {'、'.join(pm.get('authors', []))}")
48             cols[1].caption(f"期刊: {pm.get('journal', 'N/A')}")
49             cols[2].caption(f"年份: {pm.get('year', 'N/A')}")
50             if pm.get("doi"):

```

```

01         cols[3].caption(f"D01: {pm['doi']}")
02
03     # 图表分析（可选）
04     with st.expander("? 图表分析（可选）"):
05         chart_img = st.file_uploader(
06             "上传图表图片（如相关性热图、箱线图等）",
07             type=["png", "jpg", "jpeg", "gif", "webp"],
08         )
09         if chart_img:
10             image_bytes = chart_img.getvalue()
11             st.image(image_bytes, caption="已上传的图表", use_container_width=True)
12             st.session_state.chart_image_bytes = image_bytes
13             vis_key = st.text_input(
14                 "Vision API Key", type="password",
15                 key="_vision_api_key",
16                 placeholder="sk-... 或设置 BIO_LLM_API_KEY",
17             )
18             vis_url = st.text_input(
19                 "Vision Base URL", key="_vision_base_url",
20                 placeholder="https://api.openai.com/v1",
21             )
22             vis_model = st.text_input(
23                 "Vision 模型", key="_vision_model", placeholder="gpt-4o",
24             )
25             if st.button("? 解读图表", use_container_width=True):
26                 from bio_logic_debugger.llm.reasoner import LLMConfig, LLMReasoner
27                 reasoner = LLMReasoner(config=LLMConfig.from_ui(
28                     api_key=vis_key, base_url=vis_url, model=vis_model,
29                 ))
30                 reasoner.set_vision_model(vis_model or reasoner._vision_model)
31                 with st.spinner("解读图表中..."):
32                     st.session_state.chart_analysis = reasoner.analyze_vision(image_bytes)
33                 if st.session_state.get("chart_analysis"):
34                     st.markdown(st.session_state.chart_analysis)
35
36     # —— 自动检索论文 ——
37     with st.expander("? 自动检索论文（从知识库关键词搜索 CrossRef）"):
38         # 从知识库提取关键词
39         if "_search_keywords" not in st.session_state:
40             from bio_logic_debugger.knowledge.paper_search import extract_keywords_from_knowledge
41             kw = extract_keywords_from_knowledge(
42                 list(engine.iter_traits()),
43                 max_keywords=10,
44             )
45             st.session_state._search_keywords = kw
46
47         st.markdown("**检索关键词**（可编辑）：")
48         edited_keywords = st.text_input(
49             "关键词（逗号分隔）",
50             value=", ".join(st.session_state._search_keywords),

```

```

01         key="search_kw_input",
02         label_visibility="collapsed",
03     )
04     kw_list = [k.strip() for k in edited_keywords.split(",") if k.strip()]
05
06     col_s1, col_s2 = st.columns([1, 3])
07     with col_s1:
08         rows_per = st.number_input("每批数量", min_value=3, max_value=20, value=8, step=1)
09     with col_s2:
10         search_btn = st.button("? 开始检索", type="primary", use_container_width=True)
11
12     if search_btn:
13         from bio_logic_debugger.knowledge.paper_search import (
14             load_seen_papers,
15             mark_papers_seen,
16             search_all_keywords,
17         )
18         seen = load_seen_papers()
19         with st.spinner(f"正在用 {len(kw_list)} 个关键词检索 CrossRef..."):
20             papers = search_all_keywords(kw_list, rows_per_query=rows_per, seen_dois=seen)
21
22         if not papers:
23             st.info("没有找到新的论文（所有结果已检索过）")
24         else:
25             st.success(f"找到 {len(papers)} 篇新论文")
26             st.session_state._auto_search_results = papers
27
28     # 显示检索结果
29     if "_auto_search_results" in st.session_state:
30         papers = st.session_state._auto_search_results
31         if papers:
32             st.markdown("**检索结果（勾选要分析的论文）：**")
33             selected_papers = []
34             for i, p in enumerate(papers):
35                 c1, c2 = st.columns([0.05, 1])
36                 with c1:
37                     checked = st.checkbox("", key=f"paper_{i}")
38                 with c2:
39                     meta = f"**{p.get('title', '未知')}**"
40                     authors = ", ".join(p.get("authors", []))[:60]
41                     if authors:
42                         meta += f" _{authors}_ "
43                     meta += f" ({p.get('year', '?')})"
44                     if p.get("journal"):
45                         meta += f" \ {p['journal']}"
46                     if p.get("doi"):
47                         meta += f" DOI: {p['doi']}"
48                     st.markdown(meta)
49                     if p.get("abstract"):
50                         abstract_preview = p["abstract"][:200].replace("<", "&lt;").replace(">", "&g

```

```

01         st.caption(f"摘要: {abstract_preview}..." if len(p["abstract"]) > 200 else f"摘要: {
02         if checked:
03             selected_papers.append(p)
04
05         if selected_papers:
06             if st.button("? 分析选中论文", use_container_width=True):
07                 from bio_logic_debugger.knowledge.paper_search import mark_papers_seen
08                 from bio_logic_debugger.knowledge.paper_analyzer import analyze_text
09                 # 合并选中论文的摘要
10                 combined = "\n\n".join(
11                     p.get("abstract", "") for p in selected_papers
12                 )
13                 if combined.strip():
14                     with st.spinner("分析中..."):
15                         extracted = analyze_text(combined, llm_caller=None)
16                         st.session_state.extracted_items = extracted
17                         st.session_state._auto_search_results = [] # 清空结果
18                         # 标记已见
19                         mark_papers_seen(selected_papers)
20                         st.success(f"分析完成, 共提取 {len(extracted)} 条, 请在上方「分析结果」区域审核导入")
21                         st.rerun()
22                 else:
23                     st.warning("选中论文无可用摘要")
24
25     # —— 开始分析 ——
26     has_text = "paper_raw_text" in st.session_state and st.session_state.paper_raw_text.strip()
27
28     if has_text:
29         llm_for_extract = st.checkbox(
30             "启用 LLM 提取 (需配置 API Key)", value=False,
31             help="通过 LLM 提取更精确的结构化信息",
32         )
33         extract_api_key = ""
34         extract_base_url = ""
35         extract_model = ""
36         if llm_for_extract:
37             with st.expander("□□ LLM 配置", expanded=True):
38                 extract_api_key = st.text_input(
39                     "API Key", type="password",
40                     key="_extract_api_key",
41                     placeholder="sk-... 或设置 BIO_LLM_API_KEY",
42                 )
43                 extract_base_url = st.text_input(
44                     "Base URL",
45                     key="_extract_base_url",
46                     placeholder="https://api.deepseek.com/v1",
47                 )
48                 extract_model = st.text_input(
49                     "模型名", key="_extract_model", placeholder="deepseek-chat",
50                 )

```

```

01         raw_len = len(st.session_state.paper_raw_text)
02         if raw_len > 6000:
03             st.info(
04                 f"正文约 {raw_len} 字。启用 LLM 时会按约 6000 字分块送入，"
05                 "不再只取前 8000 字。"
06             )
07         analyze_btn = st.button("? 开始分析", type="primary", use_container_width=True)
08
09         if analyze_btn:
10             from bio_logic_debugger.knowledge.paper_analyzer import analyze_text
11
12             llm_caller = None
13             if llm_for_extract:
14                 from bio_logic_debugger.llm.reasoner import LLMConfig, LLMReasoner
15                 reasoner = LLMReasoner(config=LLMConfig.from_ui(
16                     api_key=extract_api_key,
17                     base_url=extract_base_url,
18                     model=extract_model,
19                 ))
20                 llm_caller = reasoner.chat
21
22             with st.spinner("分析论文中..."):
23                 extracted = analyze_text(
24                     st.session_state.paper_raw_text,
25                     llm_caller=llm_caller,
26                 )
27
28             st.session_state.extracted_items = extracted
29             st.success(f"分析完成，共提取 {len(extracted)} 条")
30
31         # 展示分析结果
32         if "extracted_items" in st.session_state and st.session_state.extracted_items:
33             items = st.session_state.extracted_items
34             st.divider()
35             st.subheader("分析结果")
36
37         # 分类展示
38         categories = {"trait": "? 性状", "correlation": "? 关联", "constraint": "? 约束"}
39         selected_ids = set()
40
41         for cat_key, cat_label in categories.items():
42             cat_items = [it for it in items if it.item_type == cat_key]
43             if not cat_items:
44                 continue
45
46             with st.expander(f"[{cat_label}] ({len(cat_items)} 条)", expanded=True):
47                 for idx, item in enumerate(cat_items):
48                     item_key = f"[{cat_key}]_{idx}"
49                     label = item.data.get("name") or item.data.get("trait_a", "未知")
50                     checked = st.checkbox(

```

```

01         f"{label} 置信度 {item.confidence:.0%}",
02         value=item.selected,
03         key=f"select_{item_key}",
04     )
05     if checked:
06         selected_ids.add(item_key)
07
08     # 显示详情
09     detail_parts = []
10     if item.data.get("trait_b"):
11         detail_parts.append(
12             f"{item.data['trait_a']} □ {item.data['trait_b']} "
13             f"类型: {item.data.get('type', '')} "
14             f"强度: {item.data.get('strength', 0)}"
15         )
16     if item.data.get("description"):
17         detail_parts.append(f"描述: {item.data['description'][:120]}")
18     if item.data.get("range"):
19         detail_parts.append(
20             f"范围: {item.data['range']} {item.data.get('unit', '')}"
21         )
22     if item.source_sentence:
23         detail_parts.append(f"原文: {item.source_sentence[:100]}")
24     if detail_parts:
25         st.markdown("\n\n".join(detail_parts))
26     st.markdown("——")
27
28 # 导入按钮
29 if st.button("? 导入到知识库", type="primary", use_container_width=True):
30     from bio_logic_debugger.knowledge.paper_analyzer import (
31         items_to_constraints,
32         items_to_correlations,
33         items_to_traits,
34     )
35
36     selected_items = []
37     for cat_key in categories:
38         cat_items = [it for it in items if it.item_type == cat_key]
39         for idx, item in enumerate(cat_items):
40             if f"{cat_key}_{idx}" in selected_ids:
41                 item.selected = True
42                 selected_items.append(item)
43
44     if not selected_items:
45         st.warning("请先勾选要导入的项")
46     else:
47         known = list(engine.iter_traits())
48         new_traits = items_to_traits(selected_items, known)
49         new_corrs = items_to_correlations(selected_items, known)
50         new_constraints = items_to_constraints(selected_items, known)

```

```

01
02         # 注册到引擎
03         from bio_logic_debugger.knowledge.knowledge_store import (
04             constraint_from_dict,
05             correlation_from_dict,
06             trait_from_dict,
07         )
08         for d in new_traits:
09             engine.register_trait(trait_from_dict(d))
10         for d in new_corrs:
11             engine.register_correlation(correlation_from_dict(d))
12         for d in new_constraints:
13             engine.register_constraint(constraint_from_dict(d))
14
15         # 保存到 session_state 用户扩充列表
16         if "_user_traits" not in st.session_state:
17             stored = knowledge_store.load_user_knowledge()
18             st.session_state._user_traits = list(stored.get("traits", []))
19             st.session_state._user_corrs = list(stored.get("correlations", []))
20             st.session_state._user_constraints = list(stored.get("constraints", []))
21             st.session_state._user_anti_patterns = list(stored.get("anti_patterns", []))
22
23         st.session_state._user_traits.extend(new_traits)
24         st.session_state._user_corrs.extend(new_corrs)
25         st.session_state._user_constraints.extend(new_constraints)
26         knowledge_store.save_user_knowledge(
27             traits=st.session_state._user_traits,
28             correlations=st.session_state._user_corrs,
29             constraints=st.session_state._user_constraints,
30         )
31
32         st.success(f"☐ 已导入 {len(selected_items)} 条到知识库（已写入用户目录）！")
33         st.rerun()
34
35     # —— Tab 2: 知识库管理 ——
36
37     with tab2:
38         st.subheader("知识库管理")
39
40         # 当前状态
41         stored_user = knowledge_store.load_user_knowledge()
42         if "_user_traits" not in st.session_state:
43             st.session_state._user_traits = list(stored_user.get("traits", []))
44             st.session_state._user_corrs = list(stored_user.get("correlations", []))
45             st.session_state._user_constraints = list(stored_user.get("constraints", []))
46
47         col1, col2, col3 = st.columns(3)
48         col1.metric("内置性状", len(knowledge_store.load_built_in().get("traits", [])), border=True)
49         col2.metric("社区性状", len(knowledge_store.load_community().get("traits", [])), border=True)
50         col3.metric("用户扩充", len(st.session_state.get("_user_traits", [])), border=True)

```

```

01
02     sync_time = knowledge_store.get_last_sync_time() or "从未同步"
03     st.caption(f"? 最后同步: {sync_time}")
04
05     # 同步按钮
06     if st.button("? 手动同步社区知识库", use_container_width=True):
07         with st.spinner("同步中..."):
08             if knowledge_store.sync_from_community():
09                 st.success("同步完成! 正在重新加载引擎...")
10                 # 清理缓存, 重新加载
11                 st.cache_resource.clear()
12                 st.rerun()
13             else:
14                 st.warning("部分同步失败, 请检查网络连接")
15
16     st.divider()
17
18     # 用户扩充列表
19     user_traits = st.session_state.get("_user_traits", [])
20     user_corrs = st.session_state.get("_user_corrs", [])
21     user_constraints = st.session_state.get("_user_constraints", [])
22
23     if user_traits or user_corrs or user_constraints:
24         st.markdown("**用户扩充的知识: **")
25
26         if user_traits:
27             with st.expander(f"? 用户性状 ({len(user_traits)} 条)"):
28                 for t in user_traits:
29                     st.markdown(f"- **{t.get('name')}** ` {t.get('id')} ` 范围: {t.get('typical_range')} {t.g
30
31         if user_corrs:
32             with st.expander(f"? 用户关联 ({len(user_corrs)} 条)"):
33                 for c in user_corrs:
34                     st.markdown(f"- {c.get('trait_a')} □ {c.get('trait_b')} 类型: {c.get('corr_type')} r={c
35
36         if user_constraints:
37             with st.expander(f"? 用户约束 ({len(user_constraints)} 条)"):
38                 for c in user_constraints:
39                     st.markdown(f"- **{c.get('name')}** 等级: {c.get('severity')}")
40
41     # 导出
42     st.divider()
43     if st.button("? 导出为 JSON", use_container_width=True):
44         exported = knowledge_store.export_user_knowledge(
45             traits=user_traits,
46             correlations=user_corrs,
47             constraints=user_constraints,
48             anti_patterns=[],
49         )
50     st.download_button(

```

```

01         "□□ 下载 JSON 文件",
02         data=exported,
03         file_name="user_knowledge_export.json",
04         mime="application/json",
05     )
06     st.info("将此 JSON 提 PR 到 TsoiZF/bio-logic-knowledge 仓库即可贡献到社区!")
07
08     else:
09         st.info("暂无用户扩充的知识。在「导入文献分析」Tab 中分析论文后导入即可。")
10
11     st.divider()
12
13     # —— 权重调整 ——
14     st.subheader("□□ 数据权重调整")
15     st.caption("调整每条知识对验证结果的影响程度。降低不可靠知识的权重可减少误报。")
16
17     user_weights = load_user_weights()
18     # tab 的内容每轮都会渲染，滑条一定进 session_state；保存时与磁盘合并。
19     wtab1, wtab2, wtab3 = st.tabs([
20         f"性状权重 ({len(engine.trait_ids())})",
21         f"关联权重 ({len(engine.iter_correlations())})",
22         f"约束权重 ({len(engine.iter_constraints())})",
23     ])
24     with wtab1:
25         st.caption("低置信度的性状，其范围越界警告将降为 INFO")
26         for t in sorted(engine.iter_traits(), key=lambda x: x.id):
27             tid = t.id
28             default_conf = user_weights.get("traits", {}).get(tid, t.confidence)
29             st.slider(
30                 f"{t.name} ({t.category})",
31                 min_value=0.0, max_value=1.0, value=float(default_conf), step=0.1,
32                 key=f"wt_trait_{tid}",
33             )
34     with wtab2:
35         st.caption("低置信度的关联，验证时有效强度降低")
36         for c in engine.iter_correlations():
37             name_a = engine.trait_name(c.trait_a)
38             name_b = engine.trait_name(c.trait_b)
39             key = f"{c.trait_a}__{c.trait_b}" if c.trait_a < c.trait_b else f"{c.trait_b}__{c.trait_a}"
40             default_conf = user_weights.get("correlations", {}).get(key, c.confidence)
41             st.slider(
42                 f"{name_a} □ {name_b} (r={c.strength})",
43                 min_value=0.0, max_value=1.0, value=float(default_conf), step=0.1,
44                 key=f"wt_corr_{key}",
45             )
46     with wtab3:
47         st.caption("低置信度的约束，违反时严重等级自动降级")
48         for c in engine.iter_constraints():
49             default_conf = user_weights.get("constraints", {}).get(c.id, c.confidence)
50             st.slider(

```

```

01         f"{c.name} ({c.severity.name}) ",
02         min_value=0.0, max_value=1.0, value=float(default_conf), step=0.1,
03         key=f"wt_cstr_{c.id} ",
04     )
05
06     if st.button("? 保存权重并重新加载引擎", type="primary", use_container_width=True):
07         save_user_weights(merge_slider_overrides(load_user_weights(), dict(st.session_state)))
08         st.cache_resource.clear()
09         st.success("☐ 权重已保存, 引擎已重新加载!")
10         st.rerun()
11
12 # —— Tab 3: 贡献指南 ——
13
14 with tab3:
15     st.subheader("? 贡献知识到社区")
16     st.markdown("""
17     本应用的知识库支持社区共享——你可以将本地提取的知识贡献到社区知识库,
18     让所有用户都能获得最新的育种知识。
19     """)
20
21     with st.container(border=True):
22         st.markdown("### 如何贡献")
23         st.markdown("""
24         1. **提取知识**: 在「导入文献分析」Tab 中上传论文, 提取性状/关联/约束
25         2. **审核勾选**: 检查提取结果, 只勾选准确、有用的条目
26         3. **导入到本地**: 点击「导入到知识库」确认导入
27         4. **导出 JSON**: 切换到「知识库管理」Tab, 点击「导出为 JSON」
28         5. **提交 PR**: 将导出的 JSON 文件提 Pull Request 到社区仓库:
29
30         ```
31         https://github.com/TsoiTZF/bio-logic-knowledge
32         ```
33         """)
34
35     with st.container(border=True):
36         st.markdown("### 社区仓库结构")
37         st.code("""
38         bio-logic-knowledge/
39         |—— traits.json          # 性状定义
40         |—— correlations.json    # 关联关系
41         |—— constraints.json     # 约束规则
42         |—— anti_patterns.json   # 反模式
43         |—— CHANGELOG.md        # 更新日志
44         """)
45
46     st.markdown("——")
47     st.markdown(
48         "? **提示**: 启动时不会自动覆盖知识库。需要更新时, 在侧边栏或本页点「检查更新 / 手动同步」。"
49     )
50

```

```

01 # ===== FILE: bio_logic_debugger/cli.py =====
02 #!/usr/bin/env python3
03 """
04 让邳城燃烧 — Bio-Logic Debugger 命令行界面
05
06 交互式育种验证控制台。运行方式：
07     python -m bio_logic_debugger.cli
08 """
09 from __future__ import annotations
10
11 import argparse
12 import logging
13 import sys
14 from typing import Optional
15
16 from bio_logic_debugger.core.domain import (
17     BreedingGoal,
18     ConstraintSeverity,
19     TraitTarget,
20 )
21 from bio_logic_debugger.core.engine import BioLogicEngine
22 from bio_logic_debugger.knowledge.knowledge_store import load_and_merge
23
24 logging.basicConfig(level=logging.WARNING, format="%(levelname)s: %(message)s")
25 logger = logging.getLogger(__name__)
26
27
28 class BioLogicCLI:
29     """交互式育种验证控制台"""
30
31     def __init__(self):
32         self.engine = BioLogicEngine()
33         self._load_default_knowledge()
34
35     def _load_default_knowledge(self) -> None:
36         traits, correlations, constraints, anti_patterns = load_and_merge()
37         self.engine.register_traits(traits)
38         self.engine.register_correlations(correlations)
39         self.engine.register_constraints(constraints)
40         self.engine.register_anti_patterns(anti_patterns)
41         logger.info(
42             "已加载 %s 个性状, %s 条关联, %s 条约束, %s 个反模式",
43             len(traits), len(correlations), len(constraints), len(anti_patterns),
44         )
45
46     # ---- 展示 ----
47
48     def list_traits(self, category: str = "") -> None:
49         """列出所有性状"""
50         traits = self.engine.iter_traits()

```

```

01         if category:
02             traits = [t for t in traits if t.category == category]
03
04         cats = {}
05         for t in traits:
06             cats.setdefault(t.category, []).append(t)
07
08         for cat, ts in cats.items():
09             print(f"\n[{cat}]")
10             for t in sorted(ts, key=lambda x: x.id):
11                 ranges = f" [{t.typical_range[0]}~{t.typical_range[1]} {t.unit}]" if t.typical_range[0] else
12                 print(f" {t.id:40s} {t.name} {ranges}")
13
14     def list_anti_patterns(self) -> None:
15         """列出所有反模式"""
16         for ap in self.engine.iter_anti_patterns():
17             severity_tag = {
18                 ConstraintSeverity.FATAL: "?",
19                 ConstraintSeverity.SEVERE: "?",
20                 ConstraintSeverity.WARNING: "?",
21                 ConstraintSeverity.INFO: "?",
22             }.get(ap.severity, "□")
23             print(f"\n{severity_tag} {ap.name}")
24             print(f" 触发性状: {' '.join(ap.trigger_traits)}")
25             print(f" 描述: {ap.description[:120]}...")
26
27     def show_trait_detail(self, trait_id: str) -> None:
28         """显示单个性状的详细信息"""
29         trait = self.engine.get_trait(trait_id)
30         if not trait:
31             print(f"未找到性状: {trait_id}")
32             return
33
34         print(f"\n{'='*50}")
35         print(f"性状: {trait.name} ({trait.id})")
36         print(f"{'='*50}")
37         print(f" 分类: {trait.category}")
38         print(f" 单位: {trait.unit}")
39         print(f" 范围: {trait.typical_range}")
40         print(f" 描述: {trait.description}")
41         if trait.tags:
42             print(f" 标签: {' '.join(trait.tags)}")
43
44         # 显示相关关联
45         print("\n 相关关联:")
46         for corr in self.engine.iter_correlations():
47             if trait.id in (corr.trait_a, corr.trait_b):
48                 other = corr.trait_b if corr.trait_a == trait.id else corr.trait_a
49                 other_name = self.engine.trait_name(other)
50                 tag = "?拮抗" if corr.is_antagonistic() else "?权衡" if corr.corr_type.name == "TRADE_OFF" else

```

```
01         print(f"      {tag} {other_name} ({corr.corr_type.name}, r={corr.strength})")
02
03     # ---- 验证 ----
04
05     def run_validation(self, goal: BreedingGoal, as_json: bool = False):
06         """执行验证并打印报告"""
07         report = self.engine.validate(goal)
08         if as_json:
09             print(report.to_json())
10             return report
11
12         print(f"\n{'='*60}")
13         print(f"  育种目标: {goal.name}")
14         print(f"  物种: {goal.species}")
15         print(f"{'='*60}")
16
17         for t in goal.targets:
18             trait = self.engine.get_trait(t.trait_id)
19             tname = trait.name if trait else t.trait_id
20             val = t.desired_value or f"[{t.range_min}~{t.range_max}]"
21             dir_symbol = {">=": "≥", "<=": "≤", "==" : "=", "range": "∈"}.get(t.direction, t.direction)
22             print(f"  {tname:20s} {dir_symbol} {val} (优先级: {t.priority})")
23
24         if goal.environment:
25             print("  环境:", goal.environment)
26
27         print()
28         print(report.narrative())
29         return report
30
31     # ---- 交互模式 ----
32
33     def interactive(self) -> None:
34         """进入交互式控制台"""
35         print("=" * 50)
36         print("  让邳城燃烧 — Bio-Logic Debugger")
37         print("  交互式育种逻辑验证控制台")
38         print("=" * 50)
39         print("\n可用命令:")
40         print("  traits [分类]    — 列出性状")
41         print("  trait <id>      — 查看性状详情")
42         print("  patterns        — 列出反模式")
43         print("  validate        — 交互式输入育种目标并验证")
44         print("  quick <trait> <dir> <val> — 快速验证: 如 quick rice_yield_per_plant >= 50")
45         print("  multi          — 多目标验证 (逐步输入)")
46         print("  help           — 帮助")
47         print("  exit          — 退出")
48
49         while True:
50             try:
```

```

01         cmd = input("\n>> ").strip()
02     except (EOFError, KeyboardInterrupt):
03         print("\n再见。")
04         break
05
06     if not cmd:
07         continue
08
09     parts = cmd.split()
10     action = parts[0].lower()
11
12     if action in ("exit", "quit", "q"):
13         print("种地去吧，育种家没有休息日。")
14         break
15
16     elif action == "traits":
17         category = " ".join(parts[1:]) if len(parts) > 1 else ""
18         self.list_traits(category)
19
20     elif action == "trait" and len(parts) > 1:
21         self.show_trait_detail(parts[1])
22
23     elif action == "patterns":
24         self.list_anti_patterns()
25
26     elif action == "validate":
27         goal = self._interactive_goal_input()
28         if goal:
29             self.run_validation(goal)
30
31     elif action == "quick" and len(parts) >= 4:
32         goal = BreedingGoal(name="快速验证", species="水稻")
33         trait_id = parts[1]
34         direction = parts[2]
35         try:
36             value = float(parts[3])
37         except ValueError:
38             print(f"无效数值: {parts[3]}")
39             continue
40         goal.add_target(TraitTarget(trait_id, desired_value=value, direction=direction))
41         self.run_validation(goal)
42
43     elif action == "multi":
44         goal = self._interactive_goal_input()
45         if goal:
46             self.run_validation(goal)
47
48     elif action == "help":
49         print("可用命令:")
50         print(" traits [分类]      — 列出性状，可选接分类筛选")

```

```

01         print(" trait <id>          — 查看性状详情")
02         print(" patterns          — 列出所有反模式")
03         print(" validate          — 交互式输入育种目标")
04         print(" quick <id> <dir> <val> — 单性状快速验证")
05         print(" exit              — 退出")
06
07     else:
08         print(f"未知命令: {action}, 输入 help 查看帮助")
09
10 def _interactive_goal_input(self) -> Optional[BreedingGoal]:
11     """交互式输入育种目标"""
12     name = input("育种目标名称: ").strip() or "未命名目标"
13     species = input("物种 (默认水稻): ").strip() or "水稻"
14
15     goal = BreedingGoal(name=name, species=species)
16     print("\n输入目标性状 (每行一个, 留空结束): ")
17     print("格式: 性状ID 方向(>=<=/=) 数值 [优先级1-10]")
18     print("示例: rice_yield_per_plant >= 50 8")
19     print("      rice_grain_length >= 7.0 6")
20     print("")
21
22     while True:
23         line = input(" > ").strip()
24         if not line:
25             break
26
27         parts = line.split()
28         if len(parts) < 3:
29             print("格式错误, 需要至少 性状ID 方向 数值")
30             continue
31
32         trait_id = parts[0]
33         direction = parts[1]
34         try:
35             value = float(parts[2])
36         except ValueError:
37             print(f"无效数值: {parts[2]}")
38             continue
39
40         priority = int(parts[3]) if len(parts) > 3 else 5
41
42         if self.engine.get_trait(trait_id) is None:
43             print(f"□ 性状 '{trait_id}' 不在知识库中, 仍将添加")
44             tname = trait_id
45         else:
46             tname = self.engine.trait_name(trait_id)
47
48         goal.add_target(TraitTarget(trait_id, desired_value=value, direction=direction, priority=priority))
49         print(f" 已添加: {tname} {direction} {value} (优先级: {priority})")
50

```

```

01         if not goal.targets:
02             print("未添加任何目标")
03             return None
04
05         context = input("\n育种背景描述（可选，留空跳过）: ").strip()
06         if context:
07             goal.context = context
08
09         return goal
10
11
12 def _parse_env_args(items: list[str]) -> dict:
13     env = {}
14     for item in items or []:
15         if "=" not in item:
16             print(f"忽略无效 --env: {item}")
17             continue
18         key, raw = item.split("=", 1)
19         key = key.strip()
20         raw = raw.strip()
21         if not key:
22             continue
23         try:
24             env[key] = float(raw)
25         except ValueError:
26             env[key] = raw
27     return env
28
29
30 def main():
31     parser = argparse.ArgumentParser(description="Bio-Logic Debugger — 育种逻辑验证系统")
32     parser.add_argument("--trait", "-t", help="查看性状详情")
33     parser.add_argument("--list-traits", "-l", action="store_true", help="列出所有性状")
34     parser.add_argument("--list-patterns", "-p", action="store_true", help="列出所有反模式")
35     parser.add_argument("--validate", "-v", nargs=3, metavar=("TRAIT", "DIR", "VAL"), help="快速验证单个目标")
36     parser.add_argument("--env", action="append", default=[], metavar="KEY=VAL", help="环境变量，可重复，如 drought_se")
37     parser.add_argument("--json", action="store_true", help="JSON 格式输出")
38     parser.add_argument("--interactive", "-i", action="store_true", help="交互模式")
39
40     args = parser.parse_args()
41     cli = BioLogicCLI()
42
43     if args.interactive or len(sys.argv) == 1:
44         cli.interactive()
45         return
46
47     if args.list_traits:
48         cli.list_traits()
49     elif args.trait:
50         cli.show_trait_detail(args.trait)

```

```

01     elif args.list_patterns:
02         cli.list_anti_patterns()
03     elif args.validate:
04         trait_id, direction, value_str = args.validate
05         try:
06             value = float(value_str)
07         except ValueError:
08             print(f"无效数值: {value_str}")
09             sys.exit(1)
10         goal = BreedingGoal(name="命令行验证", species="水稻")
11         goal.add_target(TraitTarget(trait_id, desired_value=value, direction=direction))
12         goal.environment = _parse_env_args(args.env)
13         cli.run_validation(goal, as_json=args.json)
14     else:
15         parser.print_help()
16
17
18 if __name__ == "__main__":
19     main()
20
21 # ===== FILE: bio_logic_debugger/app.py =====
22 #!/usr/bin/env python3
23 """让邳城燃烧 — Streamlit 入口。页面在 ui.pages, 引擎在 core。"""
24 from __future__ import annotations
25
26 import os
27 import sys
28
29 sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
30
31 import streamlit as st
32
33 from bio_logic_debugger.ui.pages import anti_patterns, browser, constraints, literature, validate
34 from bio_logic_debugger.ui.runtime import boot, get_engine, render_sidebar
35
36 st.set_page_config(
37     page_title="让邳城燃烧 — Bio-Logic Debugger",
38     page_icon="?",
39     layout="wide",
40     initial_sidebar_state="expanded",
41 )
42
43 boot()
44 engine = get_engine()
45 page = render_sidebar(engine)
46
47 if page == "育种目标验证":
48     validate.render(engine)
49 elif page == "性状浏览器":
50     browser.render(engine)

```

```
01 elif page == "反模式库":
02     anti_patterns.render(engine)
03 elif page == "约束规则":
04     constraints.render(engine)
05 elif page == "? 文献与知识库":
06     literature.render(engine)
07
```